

C# DO JEITO CERTO

EleMar Jr

AULA 4

DAPPER & EF CORE

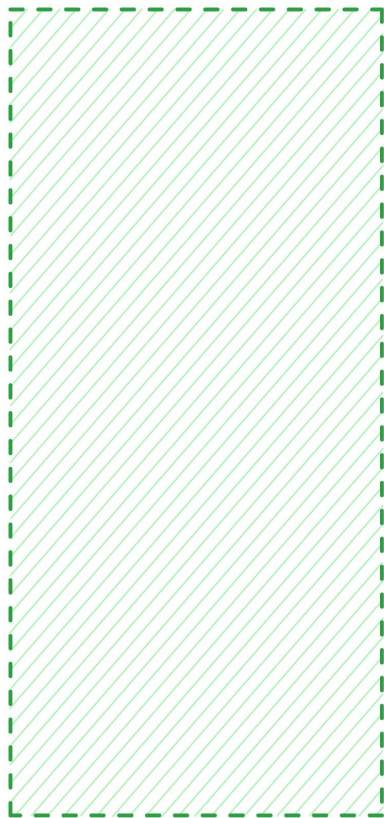


CAPÍTULO 1

ENTENDENDO O CONNECTION POOL



ACESSANDO DADOS - ENTENDENDO O CONNECTION POOL



Connection Pool

NOSSOS APLICATIVOS **ABREM E FECHAM** MUITAS **CONEXÕES IDÊNTICAS RAPIDAMENTE**

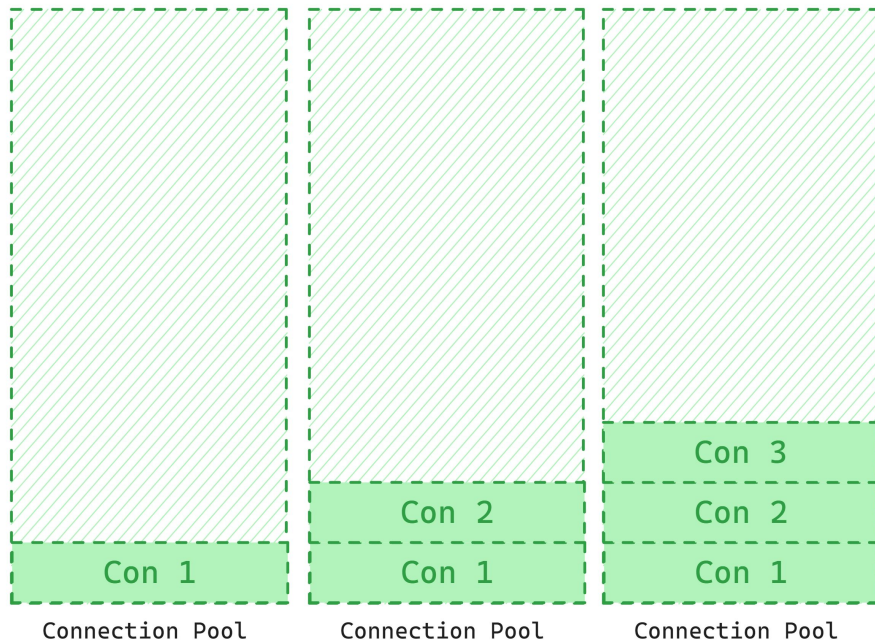
- MESMO **BANCO DE DADOS**
- MESMO **USUÁRIO**
- RESUMINDO, MESMA **CONNECTION STRING**

O **CONNECTION POOL** REDUZ O NÚMERO DE VEZES QUE UMA CONEXÃO NOVA PRECISA SER CRIADA

- MANTÉM, POR UM PERÍODO, UM **CONJUNTO DE CONEXÕES ATIVAS** PRONTAS PARA USO, POIS É PROVÁVEL QUE LOGO ELAS SEJAM UTILIZADAS NOVAMENTE

APÓS UM **TEMPO DE INATIVIDADE**, AS **CONEXÕES SÃO DESCARTADAS** E NOVAS PRECISARÃO SER CRIADAS

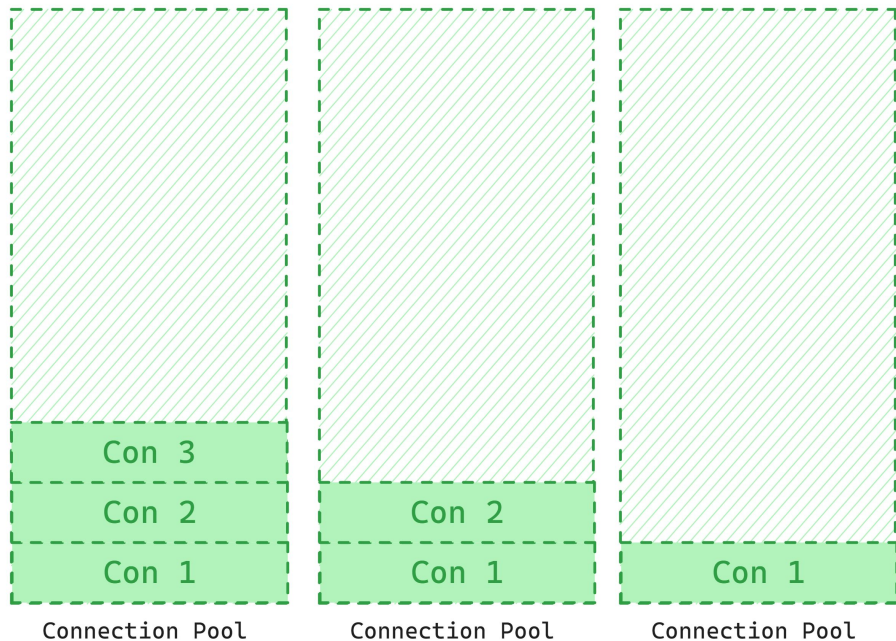
ACESSANDO DADOS - ENTENDENDO O CONNECTION POOL



CONFORME **NOVOS ACESSOS** AO BANCO DE DADOS VÃO OCORRENDO, **NOVAS CONEXÕES** VÃO SENDO ABERTAS.

O **POOL** VAI **ARMAZENANDO, MANTENDO E GERENCIANDO** MAIS CONEXÕES ATIVAS.

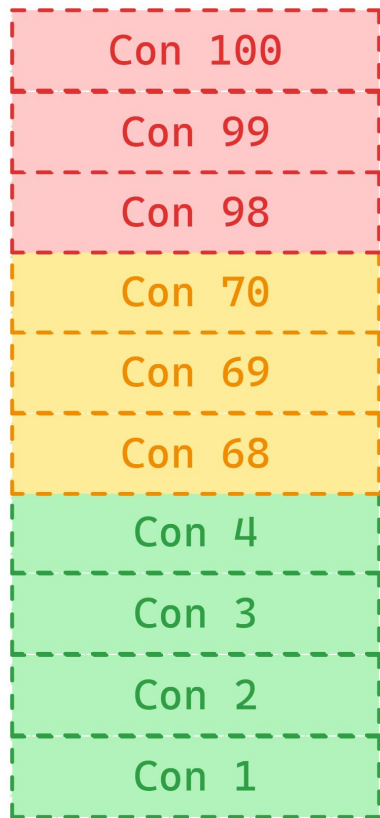
ACESSANDO DADOS - ENTENDENDO O CONNECTION POOL



E CONFORME AS **CONEXÕES** NÃO VÃO SENDO REQUERIDAS, ELAS SÃO **APROPRIADAMENTE DESCARTADAS**

TUDO ISSO OCORRE **SEM** A **NECESSIDADE DE ATENÇÃO** OU DE ALGUM **TRABALHO MANUAL**

ACESSANDO DADOS - ENTENDENDO O CONNECTION POOL



Connection Pool

O **POOL**, POR PADRÃO, COMPORTA ATÉ **100 CONEXÕES**

ACESSANDO DADOS - ENTENDENDO O CONNECTION POOL

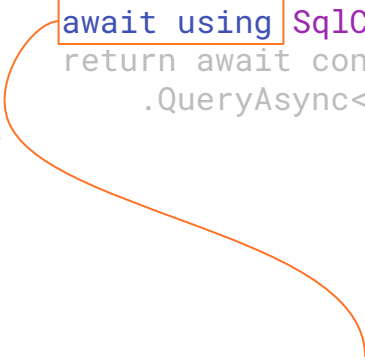
```
public async Task<IEnumerable<Something>> GetAsync(CancellationToken cancellationToken)
{
    var sql = @"<MY_SQL_CODE>";

    await using SqlConnection connection = new(_connectionString);
    return await connection
        .QueryAsync<Something>(sql);
}
```


ACESSANDO DADOS - ENTENDENDO O CONNECTION POOL

```
public async Task<IEnumerable<Something>> GetAsync(CancellationToken cancellationToken)
{
    var sql = @"<MY_SQL_CODE>";

    await using SqlConnection connection = new(_connectionString);
    return await connection
        .QueryAsync<Something>(sql);
}
```



APENAS O **USING** É O SUFICIENTE. **NÃO**
PRECISAMOS FAZER O **OPEN** NEM O
CLOSE DA **CONEXÃO**

E SE NÃO REALIZARMOS O *DISPOSE* APROPRIADAMENTE?

System.InvalidOperationException: Timeout expired. The timeout period elapsed prior to obtaining a connection from the pool. This may have occurred because all pooled connections were in use and max pool size was reached.

CAPÍTULO 2

DAPPER NA PRÁTICA



ACESSANDO DADOS - DAPPER NA PRÁTICA

```
public async Task<IEnumerable<PostDTO>> GetPostsAsync(CancellationToken cancellationToken = default)
{
    var sql = @"SELECT    Id,
                        Body,
                        CreationDate
                FROM Posts";

    await using SqlConnection connection = new(_connectionString);
    return await connection.QueryAsync<PostDTO>(sql).WaitAsync(cancellationToken).ConfigureAwait(false);
}
```

ACESSANDO DADOS - DAPPER NA PRÁTICA

```
public async Task<IEnumerable<PostDTO>> GetPostsByTitleAsync(
    string title,
    CancellationToken cancellationToken = default)
{
    var sql = @"SELECT  Id,
                        Body,
                        CreationDate
                    FROM Posts
                    WHERE Title = @title";

    await using SqlConnection connection =
        new(_connectionString);
    return await connection
        .QueryAsync<PostDTO>(sql, new { title })
        .WaitAsync(cancellationToken)
        .ConfigureAwait(false);
}
```

ACESSANDO DADOS - DAPPER NA PRÁTICA

```
exec sp_executesql N'SELECT Id,  
                        Body,  
                        CreationDate  
FROM Posts  
WHERE Title = @title',N'@title  
nvarchar(4000)',@title=N'title=What is the fastest?'
```

POR **DEFAULT**, O **DAPPER** UTILIZA
AS **STRINGS** COMO SENDO
NVARCHAR(4000)

ACESSANDO DADOS - DAPPER NA PRÁTICA

A **SETA MAIS LARGA**, NUM **PLANO DE EXECUÇÃO**, INDICA QUE HOUVE UM **SCAN** PARA A SATISFAÇÃO DAS **CLÁUSULAS**

A **QUERY ORIGINAL** FEZ O **SQL** REALIZAR UM **INDEX SCAN**



SELECT
Cost: 0 %



Clustered **Index Scan** (Clustered)
[Posts].[PK_Posts__Id]
Cost: 100 %
7.763s
0 of
187 (0%)

POR FIM, HÁ UM **ALERTA** NO **RESULT**

ACESSANDO DADOS - DAPPER NA PRÁTICA

O **CUSTO** DE UMA **QUERY**, QUANDO **MAIOR DO QUE 0**, É UM **SINAL RUIM**



SELECT	SELECT
Cost:	
Cached plan size	24 KB
Estimated Operator Cost	0,373 (0%)
Degree of Parallelism	0
Estimated Subtree Cost	629,131
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	186,88
Statement	
SELECT Id,	
	Body,
	CreationDate
FROM Posts	
WHERE Title = @title	
Warnings	
Type conversion in expression (CONVERT_IMPLICIT(nvarchar(250),[StackOverflow2010].[dbo].[Posts].[Title],0)) may affect "CardinalityEstimate" in query plan choice; Type conversion in expression (CONVERT_IMPLICIT(nvarchar(250),[StackOverflow2010].[dbo].[Posts].[Title],0)=[@title]) may affect "SeekPlan" in query plan choice	

UM **VALOR** DESSE **TAMANHO** É BASTANTE **RUIM**

Clustered Index Scan (Clustered)
[Posts].[PK_Posts__Id]

Cost: 100 %

7.763s

0 of

187 (0%)

O PRÓPRIO **SQL** NOS **ALERTA** DE QUE ELE PRECISOU REALIZAR UM **CAST** NO **DADO**

APÓS A CRIAÇÃO DE UM ÍNDICE PARA
O TÍTULO...



ACESSANDO DADOS - DAPPER NA PRÁTICA

A **QUERY** CONTINUA
REALIZANDO UM **INDEX SCAN**


SELECT
Cost: 0 %

E CONTINUAMOS COM
UM **ALERTA** NO **RESULT**


Nested Loops
(Inner Join)
Cost: 14 %
0.524s
0 of
187 (0%)

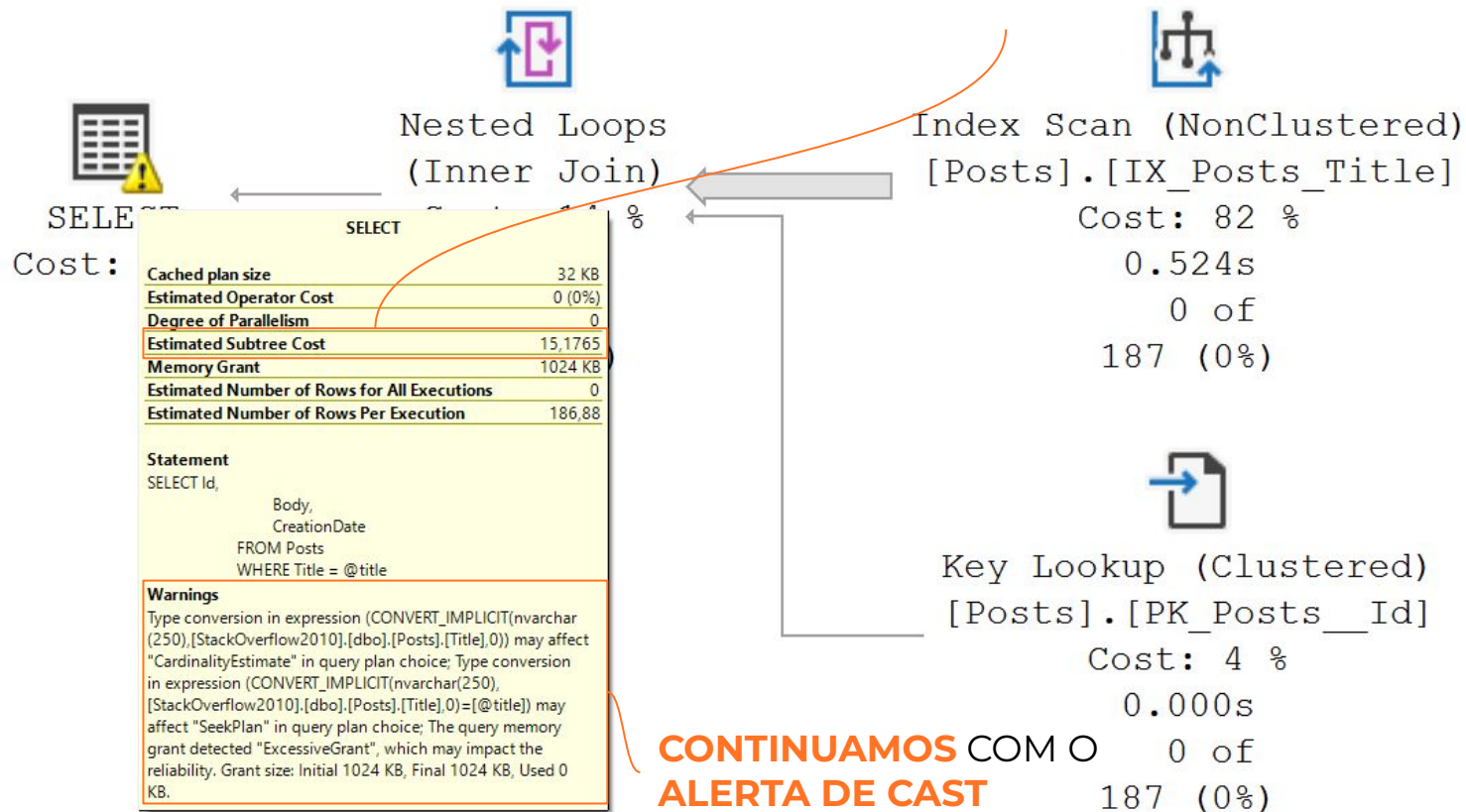

Index Scan (NonClustered)
[Posts].[IX_Posts_Title]
Cost: 82 %
0.524s
0 of
187 (0%)


Key Lookup (Clustered)
[Posts].[PK_Posts_Id]
Cost: 4 %
0.000s
0 of
187 (0%)

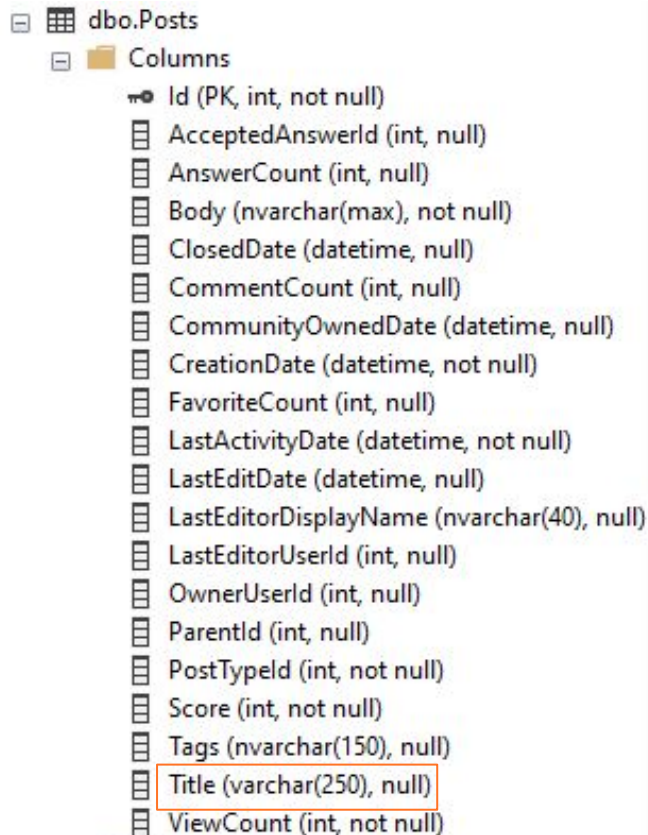
DE MODO GERAL, UM
SCAN

ACESSANDO DADOS - DAPPER NA PRÁTICA

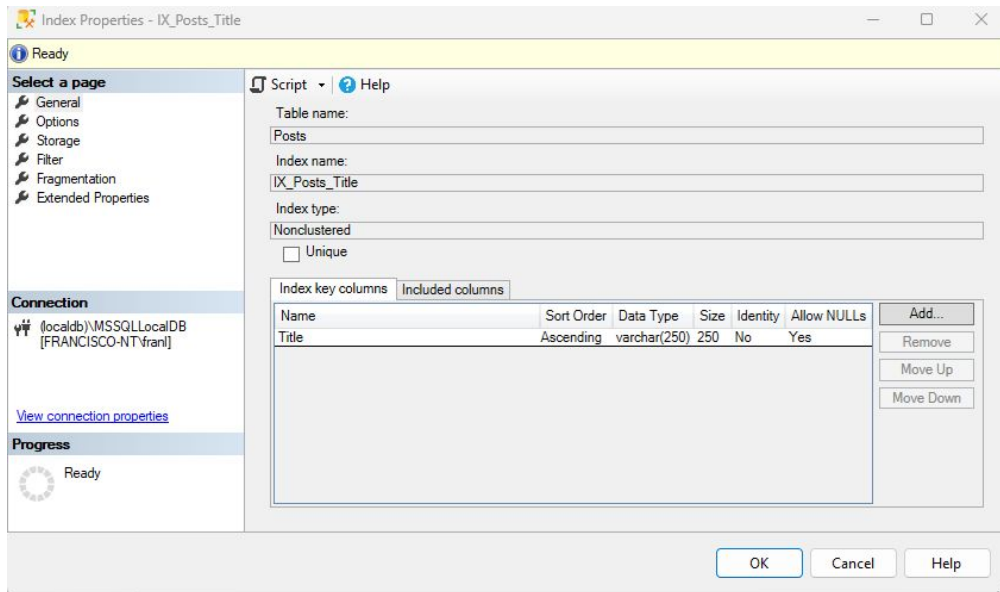
O **CUSTO DA QUERY DIMINUIU**,
MAS AINDA **NÃO É BOM**



ACESSANDO DADOS - DAPPER NA PRÁTICA



SENDO TIPOS DIFERENTES, O SQL NÃO
CONSEGUE UTILIZAR O ÍNDICE
APROPRIADAMENTE



TRABALHAR COM `DynamicParameters`
DECLARAR VARIÁVEIS NO SQL

ACESSANDO DADOS - DAPPER NA PRÁTICA

```
public async Task<IEnumerable<PostDTO>> GetPostsByTitleV2Async(
    string title,
    CancellationToken = default)
{
    var sql = @"SELECT  Id,
                        Body,
                        CreationDate
                    FROM Posts
                    WHERE Title = @title";

    DynamicParameters param = new();
    param.Add("@title", title, DbType.AnsiString, ParameterDirection.Input)

    await using SqlConnection connection = new(_connectionString);
    return await connection
        .QueryAsync<PostDTO>(sql, param)
        .WaitAsync(cancellationToken)
        .ConfigureAwait(false);
}
```

ACESSANDO DADOS - DAPPER NA PRÁTICA

```
public async Task<IEnumerable<PostDTO>> GetPostsByTitleV2Async(
    string title,
    CancellationToken = default)
{
    var sql = @"SELECT  Id,
                        Body,
                        CreationDate
                    FROM Posts
                    WHERE Title = @title";
```

UTILIZAMOS **DYNAMICPARAMETERS**
PARA **TIPIFICAR** NOSSOS
PARÂMETROS

```
DynamicParameters param = new();
param.Add("@title", title, DbType.AnsiString, ParameterDirection.Input)
```

```
await using SqlConnection connection = new(_connectionString);
return await connection
    .QueryAsync<PostDTO>(sql, param)
    .WaitAsync(cancellationToken)
    .ConfigureAwait(false);
}
```

ACESSANDO DADOS - DAPPER NA PRÁTICA

DECLARAMOS UMA **VARIÁVEL** NO **SQL**
COM O **MESMO TIPO** E **TAMANHO** DA
COLUNA QUE IREMOS FILTRAR

```
public async Task<IEnumerable<PostDTO>> GetPostsByTitleV3Async(
    string title
    CancellationToken cancellationToken = default)
{
    var sql = @"DECLARE @title_filter VARCHAR(250) = @filter;

                SELEC  Id,
                        Body,
                        CreationDate
                FROM Posts
                WHERE Title = @title_filter";

    await using SqlConnection connection = new(_connectionString);
    return await connection
        .QueryAsync<PostDTO>(sql, new { title })
        .WaitAsync(cancellationToken)
        .ConfigureAwait(false);
}
```


ACESSANDO DADOS - DAPPER NA PRÁTICA

```
public async Task<IEnumerable<PostDTO>> GetPostsByTitleV3Async(
    string title
    CancellationToken cancellationToken = default)
{
    var sql = @"DECLARE @title_filter VARCHAR(250) = @filter;

                SELEC  Id,
                        Body,
                        CreationDate
                FROM Posts
                WHERE Title = @title_filter";

    await using SqlConnection connection = new(_connectionString);
    return await connection
        .QueryAsync<PostDTO>(sql, new { title })
        .WaitAsync(cancellationToken)
        .ConfigureAwait(false);
}
```

PASSAMOS O **PARÂMETRO** SEM
NENHUMA **ESPECIFICAÇÃO** DE TIPO

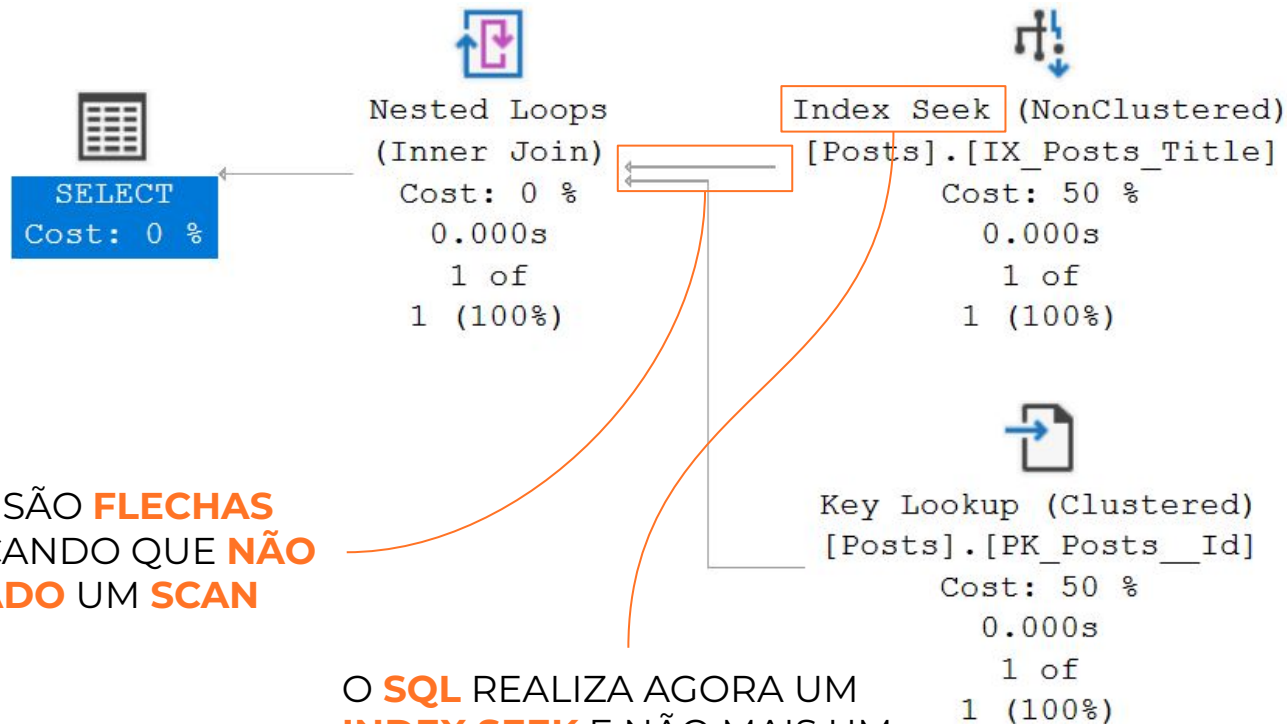
ACESSANDO DADOS - DAPPER NA PRÁTICA

```
exec sp_executesql N'SELECT      Id,  
                                Body,  
                                CreationDate  
FROM Posts  
WHERE Title = @title',N'@title  
varchar(4000)',@title='How to plot a violin scatter boxplot (in R)?'
```

PASSOU A UTILIZAR O
MESMO TIPO DA COLUNA
(**VARCHAR** E NÃO
NVARCHAR)

MESMO O **TAMANHO** CONTINUANDO COMO
4000, O **SQL** CONSEGUE FAZER **USO DO ÍNDICE**
E RODAR A **QUERY** DE FORMA **OTIMIZADA**

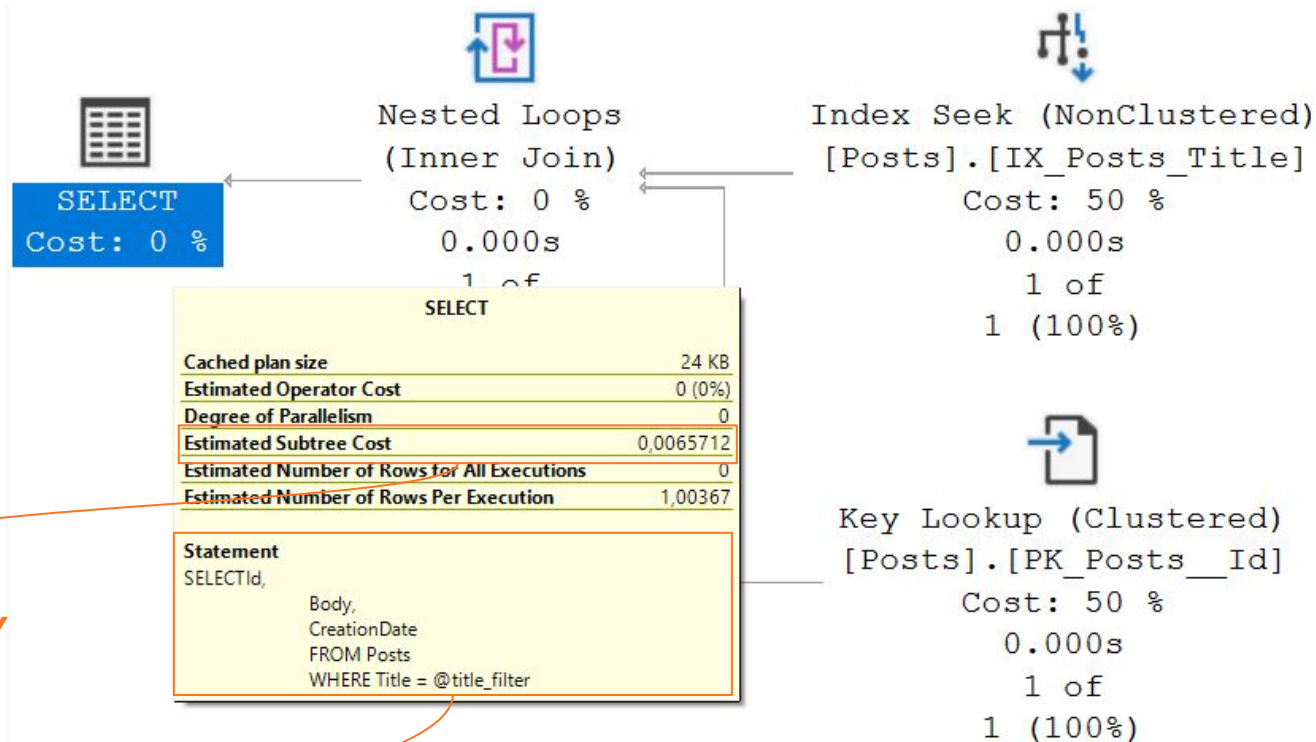
ACESSANDO DADOS - DAPPER NA PRÁTICA



AS FLECHAS SÃO **FLECHAS FINAS**, INDICANDO QUE **NÃO FOI REALIZADO** UM **SCAN**

O **SQL** REALIZA AGORA UM **INDEX SEEK** E NÃO MAIS UM **INDEX SCAN**

ACESSANDO DADOS - DAPPER NA PRÁTICA



O **CUSTO** DA
NOSSA **QUERY**
ESTÁ EM **0**

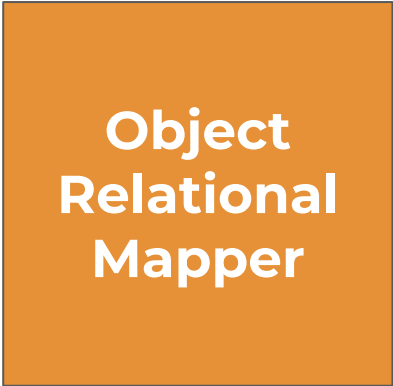
E NÃO HÁ **NENHUM**
ALERTA

CAPÍTULO 3

ENTITY FRAMEWORK NA PRÁTICA



O QUE É ENTITY FRAMEWORK CORE, NOVAMENTE?



Object
Relational
Mapper

EXECUTA E MAPEIA OS RESULTADOS DAS
QUERIES PARA **OBJETOS** E MANTÉM O **RASTREIO**
DOS SEUS **ESTADOS** PARA QUE POSSAM SER
SALVOS POSTERIORMENTE



Query
Generator

RECURSO PODEROSO PARA
GERAÇÃO DE
COMANDOS SQL

ENTITY FRAMEWORK NA PRÁTICA - CONFIGURAÇÃO DEFAULT

```
builder.Services.AddDbContext<EximiaWorkshopDbContext>(options =>
{
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"));
});
```

**Sem
retentativas**

**Todas as
queries são
rastreadas**

**Logs
limitados**

ENTITY FRAMEWORK NA PRÁTICA

```
builder.Services.AddDbContext<EximiaWorkshopDbContext>(options =>
{
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"), sqlServerOptionsAction
=>
    {
        sqlServerOptionsAction.EnableRetryOnFailure(
            maxRetryCount: 5,
            maxRetryDelay: TimeSpan.FromSeconds(1),
            errorNumbersToAdd: Enumerable.Empty<int>());
    });
});
```

CONFIGURA **RETENTATIVAS**

```
options.UseQueryTrackingBehavior(QueryTrackingBehavior.NoTracking);
```

```
// Enable just em dev
options.EnableDetailedErrors();
options.EnableSensitiveDataLogging();
```

HABILITAR OS
LOGS

ALTERA O **PADRÃO** DE
TRACKING PARA NÃO
APLICAR

APLICAR APENAS EM **AMBIENTE**
DE **DESENVOLVIMENTO**

ENTITY FRAMEWORK NA PRÁTICA

```
D:\Projects\Eximia.Course.AccessData\Eximia.Course.AccessData.API\bin\Debug\net8.0\Eximia.Course.AccessData.API.exe
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (20ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT [o].[Id], [o].[Description]
      FROM [Orders] AS [o]
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (58ms) [Parameters=[@p0='Teste Francisco' (Nullable = false) (Size = 4000)], CommandType='Text', CommandTimeout='30']
      SET IMPLICIT_TRANSACTIONS OFF;
      SET NOCOUNT ON;
      INSERT INTO [Orders] ([Description])
      OUTPUT INSERTED.[Id]
      VALUES (@p0);
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (4ms) [Parameters=[@p1='2' (Nullable = true), @p2='12.5' (Precision = 18) (Scale = 2), @p3='2' (Nullable = true), @p4='28.3' (Precision = 18) (Scale = 2)], CommandType='Text', CommandTimeout='30']
      SET IMPLICIT_TRANSACTIONS OFF;
      SET NOCOUNT ON;
      MERGE [OrderItems] USING (
      VALUES (@p1, @p2, 0),
      (@p3, @p4, 1)) AS i ([OrderId], [Value], _Position) ON 1=0
      WHEN NOT MATCHED THEN
      INSERT ([OrderId], [Value])
      VALUES (i.[OrderId], i.[Value])
      OUTPUT INSERTED.[Id], i._Position;
```

ENTITY FRAMEWORK NA PRÁTICA

NÃO DEVE SER UTILIZADO
QUANDO HÁ **ESTADO** ATRELADO
AO **DBCONTEXT**

HABILITA A UTILIZAÇÃO DO **POOL**
DE **DBCONTEXT'S**

```
builder.Services.AddDbContextPool<EximiaWorkshopDbContext>(options =>
{
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"), sqlServerOptionsAction
=>
    {
        sqlServerOptionsAction.EnableRetryOnFailure(
            maxRetryCount: 5,
            maxRetryDelay: TimeSpan.FromSeconds(1),
            errorNumbersToAdd: Enumerable.Empty<int>());
    });
});
```

ENTITY FRAMEWORK NA PRÁTICA

```
public class MeuContext : DbContext
{
    private int _tenantId; // ESTADO!

    public void SetTenant(int tenantId)
    {
        _tenantId = tenantId;
    }
}
```

UTILIZE **AsNoTracking()** NAS QUERIES CASO **NÃO DESEJE REALIZAR MODIFICAÇÕES** NOS **DADOS** RETORNADOS

UTILIZE **AsTracking()** NAS QUERIES CASO **DESEJE REALIZAR MODIFICAÇÕES** NOS **DADOS** RETORNADOS
APLICÁVEL APENAS QUANDO A **CONFIGURAÇÃO UseQueryTrackingBehavior** ESTIVER **APLICADA** PARA **NoTracking**

ENTITY FRAMEWORK - TRABALHANDO COM AsNoTracking()

SUFIXO INDICA QUE OS **OBJETOS** RETORNADOS SERÃO **SOMENTE LEITURA**

```
public async Task<IEnumerable<Order>> GetOrdersReadOnlyAsync(CancellationToken cancellationToken)
{
    return await _context
        .Orders
        .Include(o => o.Items)
        .AsNoTracking()
        .ToListAsync(cancellationToken)
        .ConfigureAwait(false);
}
```

O **AsNoTracking()** INDICA AO **CHANGE TRACKER** DO **EF** QUE OS OBJETOS RETORNADOS NÃO SERÃO MODIFICADOS, PORTANTO **NÃO SERÃO RASTREADOS**

ENTITY FRAMEWORK - TRABALHANDO COM AsTracking()

```
public async Task<IEnumerable<Order>> GetOrdersAsync(CancellationTokentoken cancellationToken)
{
    return await _context
        .Orders
        .Include(o => o.Items)
        .AsTracking()
        .ToListAsync(cancellationToken)
        .ConfigureAwait(false);
}
```

O **AsTracking()** INDICA AO **CHANGE TRACKER** DO **EF** QUE OS OBJETOS RETORNADOS PODEM SER MODIFICADOS, PORTANTO **SERÃO RASTREADOS**

ESSE É O **COMPORTAMENTO PADRÃO** DO **EF**. É PREFERÍVEL **MANTER ASSIM**, OU SEJA, **NÃO É NECESSÁRIO** ESPECIFICAR EM CADA **QUERY**

E QUANTO AO `AsNoTrackingWithIdentityResolution()`?

TAMBÉM **DESABILITA** O **RASTREAMENTO DAS ENTIDADES**

SE A **MESMA ENTIDADE** FOR **ENCONTRADA** MAIS DE UMA VEZ NA **QUERY**, A **MESMA INSTÂNCIA** SERÁ UTILIZADA

ENTITY FRAMEWORK - TRABALHANDO COM `AsNoTrackingWithIdentityResolution()`

```
var products = await _context
    .Products
    .Include(p => p.Category)
    .AsNoTrackingWithIdentityResolution()
    .ToListAsync(cancellationToken);
```

A **MESMA INSTÂNCIA** DA ENTIDADE **CATEGORY** SERÁ USADA PARA **TODOS OS PRODUTOS** QUE PERTENCEM A ESSA **CATEGORIA**

O **`AsNoTrackingWithIdentityResolution()`** INDICA AO **CHANGE TRACKER** DO **EF** QUE OS OBJETOS RETORNADOS NÃO SERÃO MODIFICADOS, PORTANTO **NÃO SERÃO RASTREADOS**

TRABALHANDO DE MANEIRA SIMPLIFICADA



ENTITY FRAMEWORK NA PRÁTICA

```
public class OrderRepositoryV1 : IOrderRepositoryV1
{
    private readonly EximiaWorkshopDbContext _context;

    public OrderRepositoryV1(EximiaWorkshopDbContext context)
    {
        _context = context;
    }

    public async Task AddAsync(Order order, CancellationToken cancellationToken)
    {
        await _context.Orders.AddAsync(order, cancellationToken).ConfigureAwait(false);
        await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }

    public async Task UpdateAsync(Order order, CancellationToken cancellationToken)
    {
        _context.Orders.Update(order);
        await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }

    public async Task DeleteAsync(int id, CancellationToken cancellationToken)
    {
        var order = await _context.Orders.FirstOrDefaultAsync(o => o.Id == id, cancellationToken).ConfigureAwait(false);
        if (order is null)
            return;

        _context.Orders.Remove(order);
        await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }
}
```

ENTITY FRAMEWORK NA PRÁTICA

```
public class OrderRepositoryV1 : IOrderRepositoryV1
{
    private readonly EximiaWorkshopDbContext _context;

    public OrderRepositoryV1(EximiaWorkshopDbContext context)
    {
        _context = context;
    }

    public async Task AddAsync(Order order, CancellationToken cancellationToken)
    {
        await _context.Orders.AddAsync(order, cancellationToken).ConfigureAwait(false);
        await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }

    public async Task UpdateAsync(Order order, CancellationToken cancellationToken)
    {
        _context.Orders.Update(order);
        await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }

    public async Task DeleteAsync(int id, CancellationToken cancellationToken)
    {
        var order = await _context.Orders.FirstOrDefaultAsync(o => o.Id == id, cancellationToken).ConfigureAwait(false);
        if (order is null)
            return;

        _context.Orders.Remove(order);
        await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }
}
```

DbContext DENTRO DO REPOSITÓRIO

ENTITY FRAMEWORK NA PRÁTICA

```
public class OrderRepositoryV1 : IOrderRepositoryV1
{
    private readonly EximiaWorkshopDbContext _context;

    public OrderRepositoryV1(EximiaWorkshopDbContext context)
    {
        _context = context;
    }

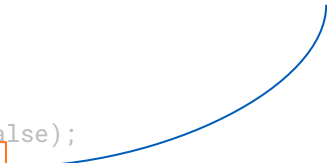
    public async Task AddAsync(Order order, CancellationToken cancellationToken)
    {
        await _context.Orders.AddAsync(order, cancellationToken).ConfigureAwait(false);
        await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }

    public async Task UpdateAsync(Order order, CancellationToken cancellationToken)
    {
        _context.Orders.Update(order);
        await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }

    public async Task DeleteAsync(int id, CancellationToken cancellationToken)
    {
        var order = await _context.Orders.FirstOrDefaultAsync(o => o.Id == id, cancellationToken).ConfigureAwait(false);
        if (order is null)
            return;

        _context.Orders.Remove(order);
        await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }
}
```

SaveChanges() DIRETAMENTE
APÓS **INSERT**



ENTITY FRAMEWORK NA PRÁTICA

```
public class OrderRepositoryV1 : IOrderRepositoryV1
{
    private readonly EximiaWorkshopDbContext _context;

    public OrderRepositoryV1(EximiaWorkshopDbContext context)
    {
        _context = context;
    }

    public async Task AddAsync(Order order, CancellationToken cancellationToken)
    {
        await _context.Orders.AddAsync(order, cancellationToken).ConfigureAwait(false);
        await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }

    public async Task UpdateAsync(Order order, CancellationToken cancellationToken)
    {
        _context.Orders.Update(order);
        await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }

    public async Task DeleteAsync(int id, CancellationToken cancellationToken)
    {
        var order = await _context.Orders.FirstOrDefaultAsync(o => o.Id == id, cancellationToken).ConfigureAwait(false);
        if (order is null)
            return;

        _context.Orders.Remove(order);
        await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }
}
```

SaveChanges() DIRETAMENTE
APÓS **UPDATE**



ENTITY FRAMEWORK NA PRÁTICA

```
public class OrderRepositoryV1 : IOrderRepositoryV1
{
    private readonly EximiaWorkshopDbContext _context;

    public OrderRepositoryV1(EximiaWorkshopDbContext context)
    {
        _context = context;
    }

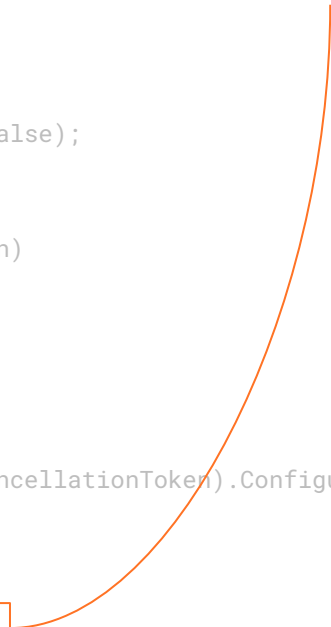
    public async Task AddAsync(Order order, CancellationToken cancellationToken)
    {
        await _context.Orders.AddAsync(order, cancellationToken).ConfigureAwait(false);
        await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }

    public async Task UpdateAsync(Order order, CancellationToken cancellationToken)
    {
        _context.Orders.Update(order);
        await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }

    public async Task DeleteAsync(int id, CancellationToken cancellationToken)
    {
        var order = await _context.Orders.FirstOrDefaultAsync(o => o.Id == id, cancellationToken).ConfigureAwait(false);
        if (order is null)
            return;

        _context.Orders.Remove(order);
        await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }
}
```

SaveChanges() DIRETAMENTE
APÓS **DELETE**



ENTITY FRAMEWORK NA PRÁTICA - COMMAND HANDLER PARA CRIAÇÃO DE UM PEDIDO

```
public class CreateOrderSimplifiedCommandHandler : IRequestHandler<CreateOrderSimplifiedCommand>
{
    private readonly IOrderRepositoryV1 _orderRepository;
    private readonly ICustomerRepositoryV1 _customerRepository;

    public CreateOrderSimplifiedCommandHandler(IOrderRepositoryV1 orderRepository, ICustomerRepositoryV1 customerRepository)
    {
        _orderRepository = orderRepository;
        _customerRepository = customerRepository;
    }

    public async Task Handle(CreateOrderSimplifiedCommand command, CancellationToken cancellationToken)
    {
        var customer = Customer.Create(command.CustomerName);
        await _customerRepository.AddAsync(customer, cancellationToken).ConfigureAwait(false);

        // Doing some work
        Thread.Sleep(2000);

        var order = Order.Create(
            command.Description,
            command.Items.Select(i => Order.Item.Create(i)).ToList(),
            customer.Id);

        await _orderRepository.AddAsync(order, cancellationToken).ConfigureAwait(false);
    }
}
```

```
public class CreateOrderSimplifiedCommandHandler : IRequestHandler<CreateOrderSimplifiedCommand>
{
    private readonly IOrderRepositoryV1 _orderRepository;
    private readonly ICustomerRepositoryV1 _customerRepository;

    public CreateOrderSimplifiedCommandHandler(IOrderRepositoryV1 orderRepository, ICustomerRepositoryV1 customerRepository)
    {
        _orderRepository = orderRepository;
        _customerRepository = customerRepository;
    }

    public async Task Handle(CreateOrderSimplifiedCommand command, CancellationToken cancellationToken)
    {
        var customer = Customer.Create(command.CustomerName);
        await _customerRepository.AddAsync(customer, cancellationToken).ConfigureAwait(false);

        // Doing some work
        Thread.Sleep(2000);

        var order = Order.Create(
            command.Description,
            command.Items.Select(i => Order.Item.Create(i)).ToList(),
            customer.Id);

        await _orderRepository.AddAsync(order, cancellationToken).ConfigureAwait(false);
    }
}
```


ENTITY FRAMEWORK NA PRÁTICA

```
public class CreateOrderSimplifiedCommandHandler : IRequestHandler<CreateOrderSimplifiedCommand>
{
    private readonly IOrderRepositoryV1 _orderRepository;
    private readonly ICustomerRepositoryV1 _customerRepository;

    public CreateOrderSimplifiedCommandHandler(IOrderRepositoryV1 orderRepository, ICustomerRepositoryV1 customerRepository)
    {
        _orderRepository = orderRepository;
        _customerRepository = customerRepository;
    }

    public async Task Handle(CreateOrderSimplifiedCommand command, CancellationToken cancellationToken)
    {
        var customer = Customer.Create(command.CustomerName);
        await _customerRepository.AddAsync(customer, cancellationToken).ConfigureAwait(false);

        // Doing some work
        Thread.Sleep(2000);

        var order = Order.Create(
            command.Description,
            command.Items.Select(i => Order.Item.Create(i)).ToList(),
            customer.Id);

        await _orderRepository.AddAsync(order, cancellationToken).ConfigureAwait(false);
    }
}
```

cria e persiste o cliente

ENTITY FRAMEWORK NA PRÁTICA

```
public class CreateOrderSimplifiedCommandHandler : IRequestHandler<CreateOrderSimplifiedCommand>
{
    private readonly IOrderRepositoryV1 _orderRepository;
    private readonly ICustomerRepositoryV1 _customerRepository;

    public CreateOrderSimplifiedCommandHandler(IOrderRepositoryV1 orderRepository, ICustomerRepositoryV1 customerRepository)
    {
        _orderRepository = orderRepository;
        _customerRepository = customerRepository;
    }

    public async Task Handle(CreateOrderSimplifiedCommand command, CancellationToken cancellationToken)
    {
        var customer = Customer.Create(command.CustomerName);
        await _customerRepository.AddAsync(customer, cancellationToken).ConfigureAwait(false);

        // Doing some work
        Thread.Sleep(2000);

        var order = Order.Create(
            command.Description,
            command.Items.Select(i => Order.Item.Create(i)).ToList(),
            customer.Id);

        await _orderRepository.AddAsync(order, cancellationToken).ConfigureAwait(false);
    }
}
```

ENTITY FRAMEWORK NA PRÁTICA

```
public class CreateOrderSimplifiedCommandHandler : IRequestHandler<CreateOrderSimplifiedCommand>
{
    private readonly IOrderRepositoryV1 _orderRepository;
    private readonly ICustomerRepositoryV1 _customerRepository;

    public CreateOrderSimplifiedCommandHandler(IOrderRepositoryV1 orderRepository, ICustomerRepositoryV1 customerRepository)
    {
        _orderRepository = orderRepository;
        _customerRepository = customerRepository;
    }

    public async Task Handle(CreateOrderSimplifiedCommand command, CancellationToken cancellationToken)
    {
        var customer = Customer.Create(command.CustomerName);
        await _customerRepository.AddAsync(customer, cancellationToken).ConfigureAwait(false);

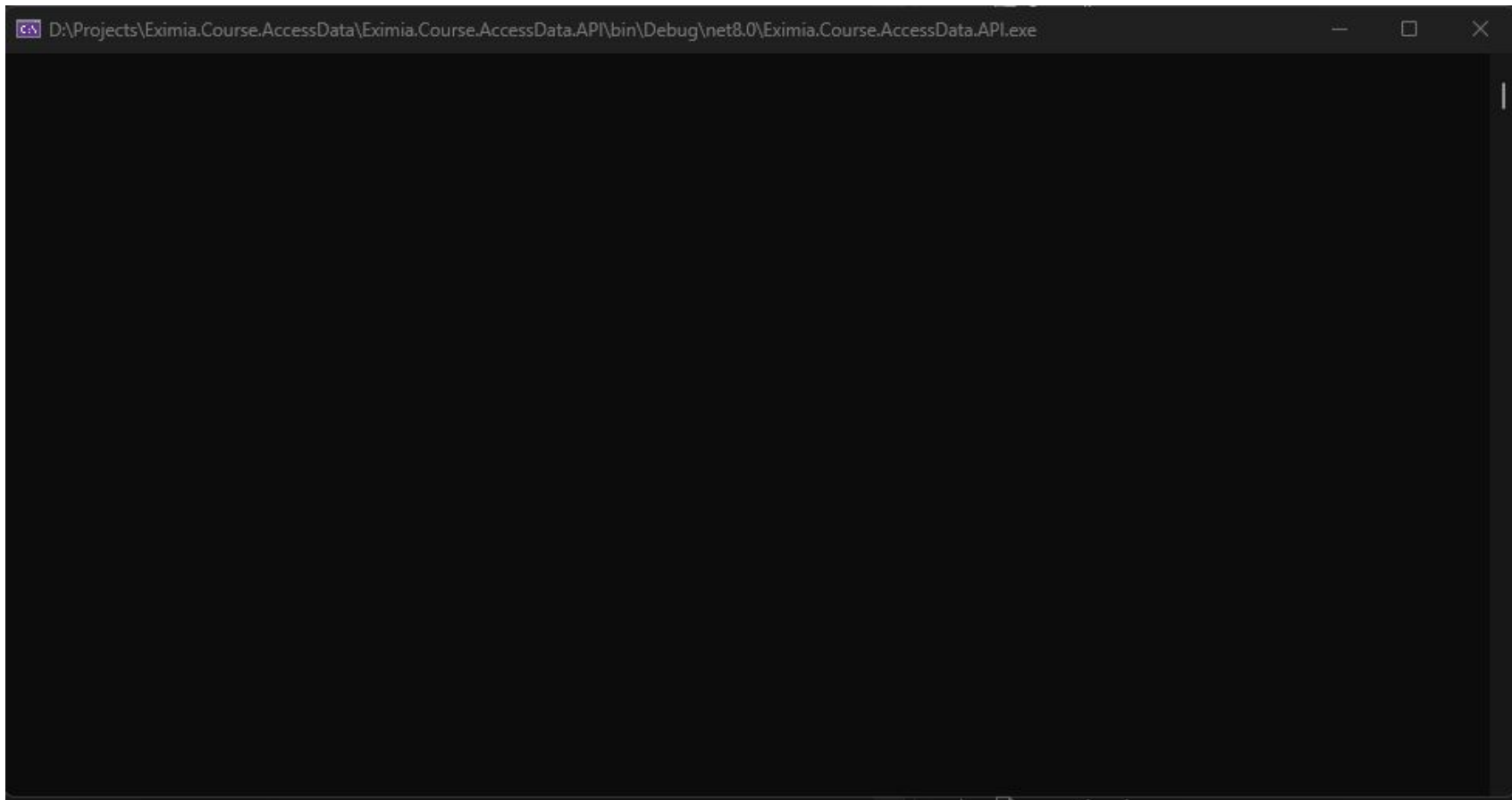
        // Doing some work
        Thread.Sleep(2000);

        var order = Order.Create(
            command.Description,
            command.Items.Select(i => Order.Item.Create(i)).ToList(),
            customer.Id);

        await _orderRepository.AddAsync(order, cancellationToken).ConfigureAwait(false);
    }
}
```

cria e persiste o cliente

ENTITY FRAMEWORK NA PRÁTICA



BAIXA CONSISTÊNCIA, UMA VEZ QUE O **COMMIT** NO BANCO DE DADOS **OCORRE SEPARADAMENTE**

DIFICULDADE NA REALIZAÇÃO DE **ROLLBACK**, VISTO QUE O DADO JÁ FOI **COMMITADO**

DEGRADAÇÃO DA **PERFORMANCE**, VISTO QUE O **NÚMERO** DE **ACIONAMENTOS** AO BANCO DE DADOS É **MAIOR**

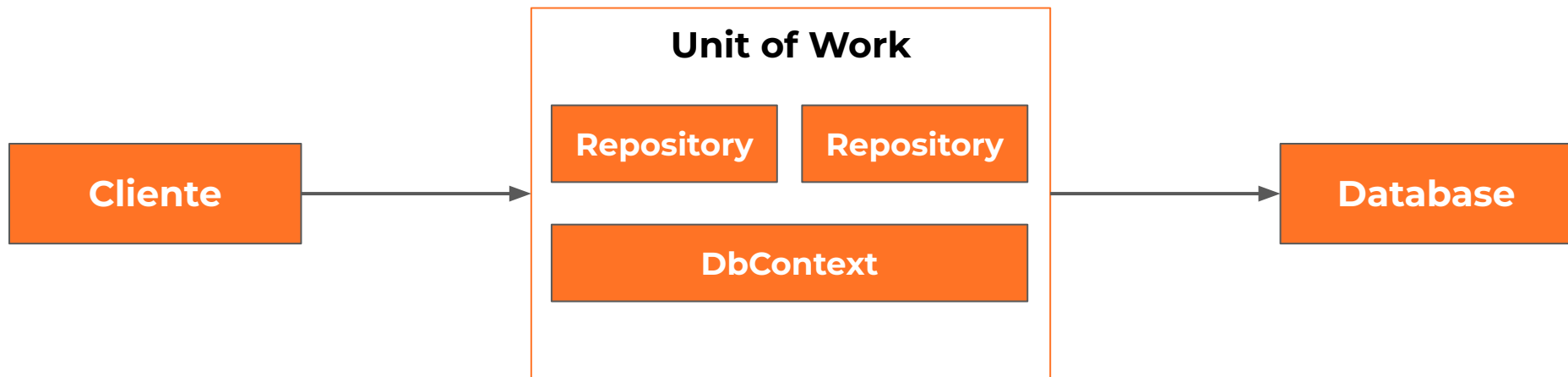
COMPLEXIFICAÇÃO DO **CÓDIGO**, VISTO QUE O DESENVOLVEDOR PRECISA SE **PREOCUPAR** COM O **CONTROLE TRANSACIONAL**

CAPÍTULO 4

UNIT OF WORK COMO SOLUÇÃO



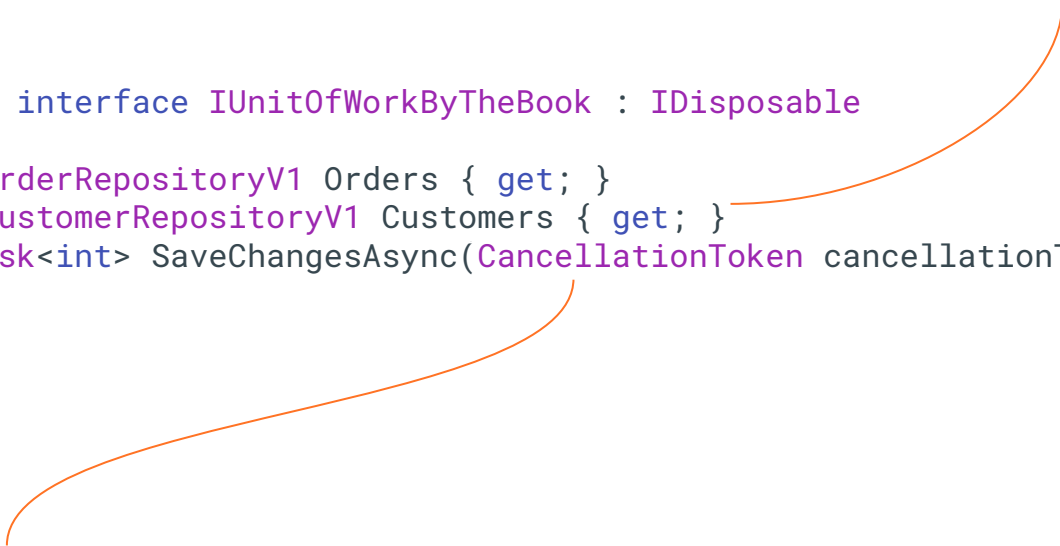
UNIT OF WORK COMO SOLUÇÃO



UNIT OF WORK COMO SOLUÇÃO - INTERFACE

ACESSO A
REPOSITÓRIOS

```
public interface IUnitOfWorkByTheBook : IDisposable
{
    IOrderRepositoryV1 Orders { get; }
    ICustomerRepositoryV1 Customers { get; }
    Task<int> SaveChangesAsync(CancellationToken cancellationToken);
}
```



MÉTODO PARA GESTÃO DAS
TRANSAÇÕES (**COMMIT**)

UNIT OF WORK COMO SOLUÇÃO

```
public class UnitOfWorkByTheBook : IUnitOfWorkByTheBook
{
    private readonly EximiaWorkshopDbContext _context;
    private bool _disposed = false;
    private IOrderRepositoryV1 _orderRepository;
    private ICustomerRepositoryV1 _customerRepository;

    public IOrderRepositoryV1 Orders => _orderRepository ??= new OrderRepositoryV1(_context);
    public ICustomerRepositoryV1 Customers => _customerRepository ??= new CustomerRepositoryV1(_context);

    public UnitOfWorkByTheBook(EximiaWorkshopDbContext context)
    {
        _context = context;
    }

    protected virtual void Dispose(bool disposing)
    {
        if (!_disposed && disposing)
        {
            _context.Dispose();
            _disposed = true;
        }
    }

    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this);
    }

    public async Task<int> SaveChangesAsync(CancellationTokens cancellationTokens)
    {
        return await _context.SaveChangesAsync(cancellationTokens).ConfigureAwait(false);
    }
}
```

UNIT OF WORK COMO SOLUÇÃO

```
public class UnitOfWorkByTheBook : IUnitOfWorkByTheBook
{
    private readonly EximiaWorkshopDbContext _context;
    private bool _disposed = false;
    private IOrderRepositoryV1 _orderRepository;
    private ICustomerRepositoryV1 _customerRepository;

    public IOrderRepositoryV1 Orders => _orderRepository ??= new OrderRepositoryV1(_context);
    public ICustomerRepositoryV1 Customers => _customerRepository ??= new CustomerRepositoryV1(_context);

    public UnitOfWorkByTheBook(EximiaWorkshopDbContext context)
    {
        _context = context;
    }

    protected virtual void Dispose(bool disposing)
    {
        if (!_disposed && disposing)
        {
            _context.Dispose();
            _disposed = true;
        }
    }

    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this);
    }

    public async Task<int> SaveChangesAsync(Cancellation_token cancellation_token)
    {
        return await _context.SaveChangesAsync(cancellation_token).ConfigureAwait(false);
    }
}
```

REPOSITÓRIOS SÃO INTERNALIZADOS
NESSA ABORDAGEM DE **UNIT OF WORK**

UNIT OF WORK COMO SOLUÇÃO

```
public class UnitOfWorkByTheBook : IUnitOfWorkByTheBook
{
    private readonly EximiaWorkshopDbContext _context;
    private bool _disposed = false;
    private IOrderRepositoryV1 _orderRepository;
    private ICustomerRepositoryV1 _customerRepository;

    public IOrderRepositoryV1 Orders => _orderRepository ??= new OrderRepositoryV1(_context);
    public ICustomerRepositoryV1 Customers => _customerRepository ??= new CustomerRepositoryV1(_context);

    public UnitOfWorkByTheBook(EximiaWorkshopDbContext context)
    {
        _context = context;
    }

    protected virtual void Dispose(bool disposing)
    {
        if (!_disposed && disposing)
        {
            _context.Dispose();
            _disposed = true;
        }
    }

    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this);
    }

    public async Task<int> SaveChangesAsync(CancellationTok... cancellationToken)
    {
        return await _context.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }
}
```

MÉTODO PARA SALVAR AS MUDANÇAS
GERENCIADAS PELO **EF**

ENTITY FRAMEWORK - COMMAND HANDLER PARA CRIAÇÃO DE UM PEDIDO COM UNIT OF WORK

```
public class CreateOrderFirstUoWCommandHandler : IRequestHandler<CreateOrderFirstUoWCommand>
{
    private readonly IUnitOfWorkByTheBook _unitOfWorkByTheBook;

    public CreateOrderFirstUoWCommandHandler(IUnitOfWorkByTheBook unitOfWorkByTheBook)
    {
        _unitOfWorkByTheBook = unitOfWorkByTheBook;
    }

    public async Task Handle(CreateOrderFirstUoWCommand command, CancellationToken cancellationToken)
    {
        var customer = Customer.Create(command.CustomerName);
        await _unitOfWorkByTheBook.Customers.AddAsync(customer, cancellationToken).ConfigureAwait(false);

        // Doing some work
        Thread.Sleep(2000);

        var order = Order.Create(
            command.Description,
            command.Items.Select(i => Order.Item.Create(i)).ToList(),
            customer.Id);

        await _unitOfWorkByTheBook.Orders.AddAsync(order, cancellationToken).ConfigureAwait(false);
        await _unitOfWorkByTheBook.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }
}
```

ENTITY FRAMEWORK - COMMAND HANDLER PARA CRIAÇÃO DE UM PEDIDO COM UNIT OF WORK

INJEÇÃO DO UNIT OF WORK

```
public class CreateOrderFirstUoWCommandHandler : IRequestHandler<CreateOrderFirstUoWCommand>
{
    private readonly IUnitOfWorkByTheBook _unitOfWorkByTheBook;

    public CreateOrderFirstUoWCommandHandler(IUnitOfWorkByTheBook unitOfWorkByTheBook)
    {
        _unitOfWorkByTheBook = unitOfWorkByTheBook;
    }

    public async Task Handle(CreateOrderFirstUoWCommand command, CancellationToken cancellationToken)
    {
        var customer = Customer.Create(command.CustomerName);
        await _unitOfWorkByTheBook.Customers.AddAsync(customer, cancellationToken).ConfigureAwait(false);

        // Doing some work
        Thread.Sleep(2000);

        var order = Order.Create(
            command.Description,
            command.Items.Select(i => Order.Item.Create(i)).ToList(),
            customer.Id);

        await _unitOfWorkByTheBook.Orders.AddAsync(order, cancellationToken).ConfigureAwait(false);
        await _unitOfWorkByTheBook.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }
}
```

ENTITY FRAMEWORK - COMMAND HANDLER PARA CRIAÇÃO DE UM PEDIDO COM UNIT OF WORK

```
public class CreateOrderFirstUoWCommandHandler : IRequestHandler<CreateOrderFirstUoWCommand>
{
    private readonly IUnitOfWorkByTheBook _unitOfWorkByTheBook;

    public CreateOrderFirstUoWCommandHandler(IUnitOfWorkByTheBook unitOfWorkByTheBook)
    {
        _unitOfWorkByTheBook = unitOfWorkByTheBook;
    }

    public async Task Handle(CreateOrderFirstUoWCommand command, CancellationToken cancellationToken)
    {
        var customer = Customer.Create(command.CustomerName);
        await _unitOfWorkByTheBook.Customers.AddAsync(customer, cancellationToken).ConfigureAwait(false);

        // Doing some work
        Thread.Sleep(2000);

        var order = Order.Create(
            command.Description,
            command.Items.Select(i => Order.Item.Create(i)).ToList(),
            customer.Id);

        await _unitOfWorkByTheBook.Orders.AddAsync(order, cancellationToken).ConfigureAwait(false);
        await _unitOfWorkByTheBook.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }
}
```

**CRIA E PERSISTE O
CLIENTE DA MESMA
FORMA**

ENTITY FRAMEWORK - COMMAND HANDLER PARA CRIAÇÃO DE UM PEDIDO COM UNIT OF WORK

```
public class CreateOrderFirstUoWCommandHandler : IRequestHandler<CreateOrderFirstUoWCommand>
{
    private readonly IUnitOfWorkByTheBook _unitOfWorkByTheBook;

    public CreateOrderFirstUoWCommandHandler(IUnitOfWorkByTheBook unitOfWorkByTheBook)
    {
        _unitOfWorkByTheBook = unitOfWorkByTheBook;
    }

    public async Task Handle(CreateOrderFirstUoWCommand command, CancellationToken cancellationToken)
    {
        var customer = Customer.Create(command.CustomerName);
        await _unitOfWorkByTheBook.Customers.AddAsync(customer, cancellationToken).ConfigureAwait(false);

        // Doing some work
        Thread.Sleep(2000);

        var order = Order.Create(
            command.Description,
            command.Items.Select(i => Order.Item.Create(i)).ToList(),
            customer.Id);

        await _unitOfWorkByTheBook.Orders.AddAsync(order, cancellationToken).ConfigureAwait(false);
        await _unitOfWorkByTheBook.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }
}
```

ENTITY FRAMEWORK - COMMAND HANDLER PARA CRIAÇÃO DE UM PEDIDO COM UNIT OF WORK

```
public class CreateOrderFirstUoWCommandHandler : IRequestHandler<CreateOrderFirstUoWCommand>
{
    private readonly IUnitOfWorkByTheBook _unitOfWorkByTheBook;

    public CreateOrderFirstUoWCommandHandler(IUnitOfWorkByTheBook unitOfWorkByTheBook)
    {
        _unitOfWorkByTheBook = unitOfWorkByTheBook;
    }

    public async Task Handle(CreateOrderFirstUoWCommand command, CancellationToken cancellationToken)
    {
        var customer = Customer.Create(command.CustomerName);
        await _unitOfWorkByTheBook.Customers.AddAsync(customer, cancellationToken).ConfigureAwait(false);

        // Doing some work
        Thread.Sleep(2000);

        var order = Order.Create(
            command.Description,
            command.Items.Select(i => Order.Item.Create(i)).ToList(),
            customer.Id);

        await _unitOfWorkByTheBook.Orders.AddAsync(order, cancellationToken).ConfigureAwait(false);
        await _unitOfWorkByTheBook.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }
}
```

**CRIA E PERSISTE O
PEDIDO DA MESMA
FORMA**

ENTITY FRAMEWORK - COMMAND HANDLER PARA CRIAÇÃO DE UM PEDIDO COM UNIT OF WORK

```
public class CreateOrderFirstUoWCommandHandler : IRequestHandler<CreateOrderFirstUoWCommand>
{
    private readonly IUnitOfWorkByTheBook _unitOfWorkByTheBook;

    public CreateOrderFirstUoWCommandHandler(IUnitOfWorkByTheBook unitOfWorkByTheBook)
    {
        _unitOfWorkByTheBook = unitOfWorkByTheBook;
    }

    public async Task Handle(CreateOrderFirstUoWCommand command, CancellationToken cancellationToken)
    {
        var customer = Customer.Create(command.CustomerName);
        await _unitOfWorkByTheBook.Customers.AddAsync(customer, cancellationToken).ConfigureAwait(false);

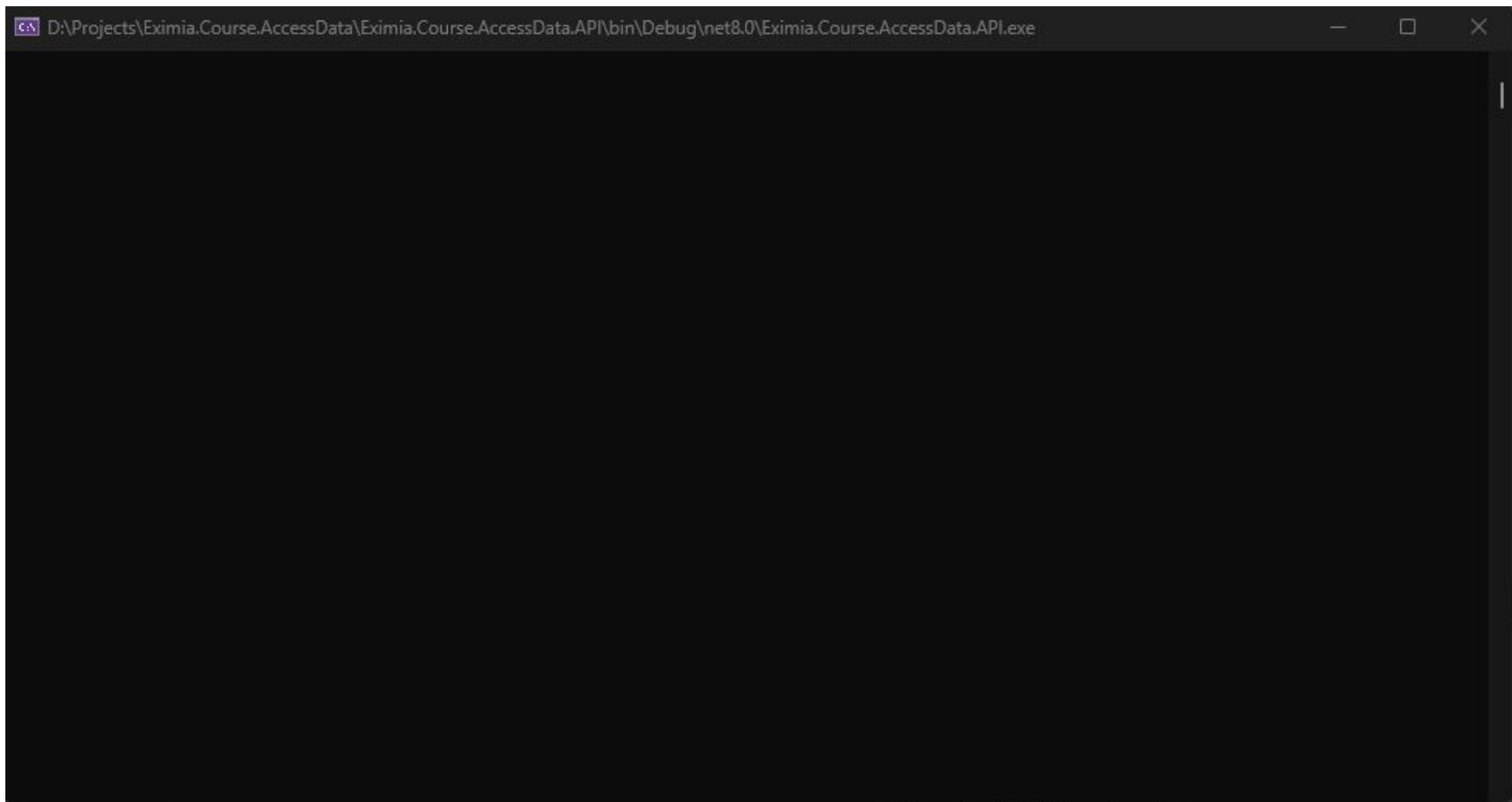
        // Doing some work
        Thread.Sleep(2000);

        var order = Order.Create(
            command.Description,
            command.Items.Select(i => Order.Item.Create(i)).ToList(),
            customer.Id);

        await _unitOfWorkByTheBook.Orders.AddAsync(order, cancellationToken).ConfigureAwait(false);
        await _unitOfWorkByTheBook.SaveChangesAsync(cancellationToken).ConfigureAwait(false);
    }
}
```

FAZ O **COMMIT** NO
BANCO DE DADOS

ENTITY FRAMEWORK DO JEITO CERTO



UMA FORMA ALTERNATIVA PARA TRABALHAR COM O UNIT OF WORK



ENTITY FRAMEWORK - FORMA ALTERNATIVA PARA SE TRABALHAR COM UNIT OF WORK

O **DBCONTEXT** É O
PRÓPRIO **UNIT OF
WORK**

```
public class EximiaWorkshopDbContext : DbContext, IUnitOfWork
{
    public EximiaWorkshopDbContext(DbContextOptions options) : base(options) { }

    public DbSet<Order> Orders { get; set; } = null!;
    public DbSet<Customer> Customers { get; set; } = null!;

    public async Task<bool> SaveEntitiesAsync(CancellationToken cancellationTok = default)
    {
        // Aqui você pode adicionar lógica extra (Domain Events, auditoria, etc)
        var result = await base.SaveChangesAsync(cancellationTok).ConfigureAwait(false);
        return result > 0;
    }
}
```

O **COMMIT**, CONSEQUENTEMENTE,
FICA NO **DBCONTEXT**

ENTITY FRAMEWORK - FORMA ALTERNATIVA PARA SE TRABALHAR COM UNIT OF WORK

```
public class OrderRepositoryV3 : IOrderRepositoryV3
{
    private readonly EximiaWorkshopDbContext _context;

    public OrderRepositoryV3(EximiaWorkshopDbContext context)
    {
        _context = context;
    }

    public IUnitOfWork UnitOfWork => _context;

    public async Task AddAsync(Order order, CancellationToken cancellationToken)
    {
        await _context
            .Orders
            .AddAsync(order, cancellationToken)
            .ConfigureAwait(false);
    }
}
```

ENTITY FRAMEWORK - FORMA ALTERNATIVA PARA SE TRABALHAR COM UNIT OF WORK

```
public class CreateOrderSecondUoWCommandHandler : IRequestHandler<CreateOrderSecondUoWCommand>
{
    private readonly ICustomerRepositoryV3 _customerRepository;
    private readonly IOrderRepositoryV3 _orderRepository;

    public CreateOrderSecondUoWCommandHandler(
        ICustomerRepositoryV3 customerRepository,
        IOrderRepositoryV3 orderRepository)
    {
        _customerRepository = customerRepository;
        _orderRepository = orderRepository;
    }

    public async Task Handle(CreateOrderSecondUoWCommand command, CancellationToken cancellationToken)
    {
        var customer = Customer.Create(command.CustomerName);
        await _customerRepository.AddAsync(customer, cancellationToken).ConfigureAwait(false);

        // Doing some work
        Thread.Sleep(2000);

        var order = Order.Create(
            command.Description,
            command.Items.Select(i => Order.Item.Create(i)).ToList(),
            customer.Id);

        await _orderRepository.AddAsync(order, cancellationToken).ConfigureAwait(false);
        await _orderRepository.UnitOfWork.SaveEntitiesAsync(cancellationToken).ConfigureAwait(false);
    }
}
```

ENTITY FRAMEWORK - FORMA ALTERNATIVA PARA SE TRABALHAR COM UNIT OF WORK

```
public class CreateOrderSecondUoWCommandHandler : IRequestHandler<CreateOrderSecondUoWCommand>
{
    private readonly ICustomerRepositoryV3 _customerRepository;
    private readonly IOrderRepositoryV3 _orderRepository;

    public CreateOrderSecondUoWCommandHandler(
        ICustomerRepositoryV3 customerRepository,
        IOrderRepositoryV3 orderRepository)
    {
        _customerRepository = customerRepository;
        _orderRepository = orderRepository;
    }

    public async Task Handle(CreateOrderSecondUoWCommand command, CancellationToken cancellationToken)
    {
        var customer = Customer.Create(command.CustomerName);
        await _customerRepository.AddAsync(customer, cancellationToken).ConfigureAwait(false);

        // Doing some work
        Thread.Sleep(2000);

        var order = Order.Create(
            command.Description,
            command.Items.Select(i => Order.Item.Create(i)).ToList(),
            customer.Id);

        await _orderRepository.AddAsync(order, cancellationToken).ConfigureAwait(false);
        await _orderRepository.UnitOfWork.SaveEntitiesAsync(cancellationToken).ConfigureAwait(false);
    }
}
```

REALIZA O **COMMIT** ATRAVÉS DO **UNIT OF WORK** QUE JÁ ESTÁ NO **REPOSITÓRIO**

UNIT OF WORK

O **UNIT OF WORK** POSSUI UM PROPÓSITO: GARANTIR QUE MÚLTIPLAS OPERAÇÕES SEJAM TRATADAS COMO UMA ÚNICA **TRANSAÇÃO ATÔMICA**

EXISTEM **DIVERSAS IMPLEMENTAÇÕES** DESSE **PADRÃO**. A **PREMISSA ACIMA**, CONTUDO, **DEVE SER PRESERVADA**

IMPLICAÇÕES DO PADRÃO BEM ADOTADO:

- CONSISTÊNCIA DOS DADOS
- POSSIBILIDADE DE ROLLBACK
- SIMPLIFICAÇÃO DO CÓDIGO
- MAIOR PERFORMANCE

MAPEANDO AS PROPRIEDADES



MAPEANDO AS PROPRIEDADES

MAPEARÁ PARA UMA **TABELA**
CHAMADA **Person**

POR CONVENÇÃO, O **EF** CONSEGUE
MAPEAR AS **PROPRIEDADES** QUE
POSSUÍREM UM **SETTER**, MESMO
QUE PRIVADO

```
public class Person
{
    public int Id { get; set; }
    public string Name { get; set; }
    public DateTime DateOfBirth { get; }
}
```

A PROPRIEDADE **DateOfBirth**
NÃO POSSUI SETTER, OU SEJA,
POR PADRÃO **NÃO SERÁ**
MAPEADA

MAPEARÁ PARA **COLONAS**
COM ESSES **NOMES E TIPOS**

MAPEANDO AS PROPRIEDADES

MAPEARÁ PARA UMA
TABELA CHAMADA **PEOPLE**

INDICA QUE É **PK** DA
TABELA

```
[Table("People")]
public class Person
{
    [Key]
    public int Id { get; set; }

    [Required]
    public string Name { get; set; }
    public DateTime DateOfBirth { get; }
}
```

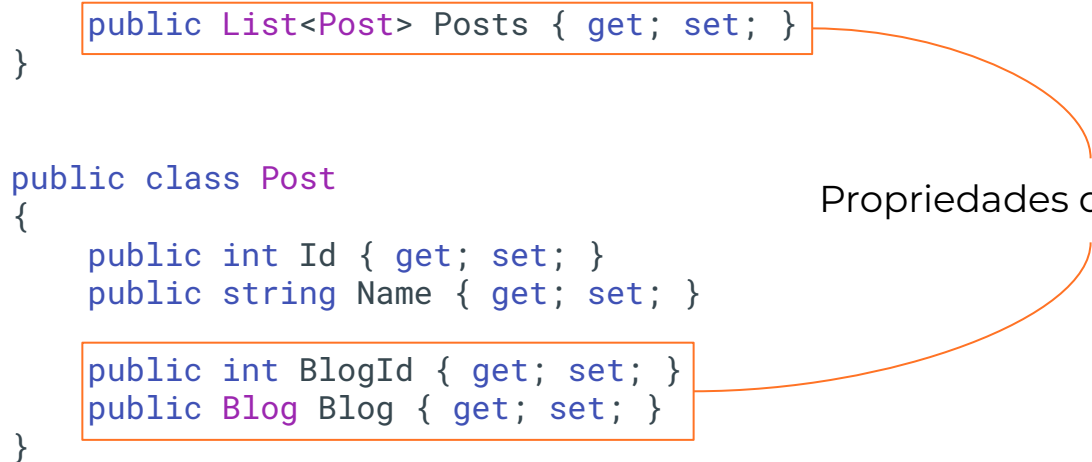
INDICA QUE O **CAMPO** É
OBRIGATÓRIO

MAPEANDO AS PROPRIEDADES

```
public class Blog
{
    public int Id { get; set; }
    public string Name { get; set; }
    public List<Post> Posts { get; set; }
}
```

```
public class Post
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int BlogId { get; set; }
    public Blog Blog { get; set; }
}
```

Propriedades de navegação



FÁCIL COMPREENSÃO E UTILIZAÇÃO PORÉM **INVADE** O DOMÍNIO COM **CONFIGURAÇÕES** DE BANCO DE DADOS

MAS COMO IR ALÉM DA CONVENÇÃO SEM INVADIR O DOMÍNIO?

A RESPOSTA É O USO DA **FLUENT API** DO **EF CORE**

MAPEANDO AS PROPRIEDADES

```
public class Signature
{
    private ICollection<Beneficiary> _beneficiaries;

    // Necessário para o EF Core
    private Signature(BankData? bankData)
    {
        BankData = bankData;
    }

    public Signature(int id, int contractId, string description, Validity validity,
        ESignatureStatus status, Guid subscriberId, Guid holderId, string externalSignatureNumber,
        int actualNumberOfBeneficiaries, ICollection<Beneficiary> beneficiaries)
    {
        ContractId = contractId;
        Description = description;
        Validity = validity;
        Status = status;
        HolderId = holderId;
        ExternalSignatureNumber = externalSignatureNumber;
        _beneficiaries = beneficiaries ?? new List<Beneficiary>();
    }

    public int Id { get; }
    public int ContractId { get; }
    public string Description { get; }
    public Validity Validity { get; }
    public ESignatureStatus Status { get; }
    public Guid HolderId { get; }
    public string ExternalSignatureNumber { get; }
    public BankData? BankData { get; }
    public IEnumerable<Beneficiary> Beneficiaries => _beneficiaries;
}
```

MAPEANDO AS PROPRIEDADES

```
public class Signature
{
    private ICollection<Beneficiary> _beneficiaries;

    // Necessário para o EF Core
    private Signature(BankData? bankData)
    {
        BankData = bankData;
    }

    public Signature(int id, int contractId, string description, Validity validity,
        ESignatureStatus status, Guid subscriberId, Guid holderId, string externalSignatureNumber,
        int actualNumberOfBeneficiaries, ICollection<Beneficiary> beneficiaries)
    {
        ContractId = contractId;
        Description = description;
        Validity = validity;
        Status = status;
        HolderId = holderId;
        ExternalSignatureNumber = externalSignatureNumber;
        _beneficiaries = beneficiaries ?? new List<Beneficiary>();
    }

    public int Id { get; }
    public int ContractId { get; }
    public string Description { get; }
    public Validity Validity { get; }
    public ESignatureStatus Status { get; }
    public Guid HolderId { get; }
    public string ExternalSignatureNumber { get; }
    public BankData? BankData { get; }
    public IEnumerable<Beneficiary> Beneficiaries => _beneficiaries;
}
```

Value Object

Enum

Value Object que
é persistido
criptografado

Lista de outro
objeto complexo

MAPEANDO AS PROPRIEDADES - COMO NÃO FAZER

```
144 > 0 references | 0 exceptions, - live  
622 protected override void OnModelCreating(DbModelBuilder modelBuilder)...
```

MÉTODO COM 475 LINHAS

MAPEANDO AS PROPRIEDADES

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.ApplyConfigurationsFromAssembly(typeof(EximiaWorkshopDbContext).Assembly);
}
```

MAPEANDO AS PROPRIEDADES - COMO NÃO FAZER

```
public class SignaturesMap : IEntityTypeConfiguration<Signature>
{
    public void Configure(EntityTypeBuilder<Signature> builder)
    {
        builder.ToTable("Signatures");
        builder.HasKey(s => s.Id);

        builder.Property(s => s.ContractId).IsRequired();
        builder.Property(s => s.Description).HasColumnType("varchar(100)").IsRequired();
        builder.Property(s => s.Status)
            .HasConversion(new EnumToStringConverter<ESignatureStatus>())
            .HasColumnType("varchar(30)")
            .IsRequired();
        builder.Property(s => s.HolderId).IsRequired();
        builder.Property(s => s.ExternalSignatureNumber).HasColumnType("varchar(100)").IsRequired(false);
        builder.Property(b => b.BankData)
            .HasConversion(
                c => c == null ? null : CryptoHelper.TripleDesEncrypt(c.ToJson()),
                s => string.IsNullOrEmpty(s) ? null! : CryptoHelper.TripleDesDecrypt(s).ToObject<BankData>())
            .HasColumnType("varchar(500)")
            .IsRequired(false);
    }
}
```

MAPEANDO AS PROPRIEDADES

```
public class SignaturesMap : IEntityTypeConfiguration<Signature>
{
    public void Configure(EntityTypeBuilder<Signature> builder)
    {
        builder.ToTable("Signatures");
        builder.HasKey(s => s.Id);

        builder.Property(s => s.ContractId).IsRequired();
        builder.Property(s => s.Description).HasColumnType("varchar(100)").IsRequired();
        builder.Property(s => s.Status)
            .HasConversion(new EnumToStringConverter<ESignatureStatus>())
            .HasColumnType("varchar(30)")
            .IsRequired();
        builder.Property(s => s.HolderId).IsRequired();
        builder.Property(s => s.ExternalSignatureNumber).HasColumnType("varchar(100)").IsRequired(false);
        builder.Property(b => b.BankData)
            .HasConversion(
                c => c == null ? null : CryptoHelper.TripleDesEncrypt(c.ToJson()),
                s => string.IsNullOrEmpty(s) ? null! : CryptoHelper.TripleDesDecrypt(s).ToObject<BankData>())
            .HasColumnType("varchar(500)")
            .IsRequired(false);
    }
}
```

IMPLEMENTA A **INTERFACE** DO **EF CORE** INFORMANDO A **ENTIDADE** A SER **MAPEADA**. NESSE CASO, A **SIGNATURE**

MAPEANDO AS PROPRIEDADES

```
public class SignaturesMap : IEntityTypeConfiguration<Signature>
{
    public void Configure(EntityTypeBuilder<Signature> builder)
    {
        builder.ToTable("Signatures");
        builder.HasKey(s => s.Id);

        builder.Property(s => s.ContractId).IsRequired();
        builder.Property(s => s.Description).HasColumnType("varchar(100)").IsRequired();
        builder.Property(s => s.Status)
            .HasConversion(new EnumToStringConverter<ESignatureStatus>())
            .HasColumnType("varchar(30)")
            .IsRequired();
        builder.Property(s => s.HolderId).IsRequired();
        builder.Property(s => s.ExternalSignatureNumber).HasColumnType("varchar(100)").IsRequired(false);
        builder.Property(b => b.BankData)
            .HasConversion(
                c => c == null ? null : CryptoHelper.TripleDesEncrypt(c.ToJson()),
                s => string.IsNullOrEmpty(s) ? null! : CryptoHelper.TripleDesDecrypt(s).ToObject<BankData>())
            .HasColumnType("varchar(500)")
            .IsRequired(false);
    }
}
```

MAPEIA A **TABELA**. PODE
SER UM NOME
QUALQUER, **DIFERINDO**
INCLUSIVE DA **CLASSE**

MAPEANDO AS PROPRIEDADES

```
public class SignaturesMap : IEntityTypeConfiguration<Signature>
{
    public void Configure(EntityTypeBuilder<Signature> builder)
    {
        builder.ToTable("Signatures");
        builder.HasKey(s => s.Id);

        builder.Property(s => s.ContractId).IsRequired();
        builder.Property(s => s.Description).HasColumnType("varchar(100)").IsRequired();
        builder.Property(s => s.Status)
            .HasConversion(new EnumToStringConverter<ESignatureStatus>())
            .HasColumnType("varchar(30)")
            .IsRequired();
        builder.Property(s => s.HolderId).IsRequired();
        builder.Property(s => s.ExternalSignatureNumber).HasColumnType("varchar(100)").IsRequired(false);
        builder.Property(b => b.BankData)
            .HasConversion(
                c => c == null ? null : CryptoHelper.TripleDesEncrypt(c.ToJson()),
                s => string.IsNullOrEmpty(s) ? null! : CryptoHelper.TripleDesDecrypt(s).ToObject<BankData>())
            .HasColumnType("varchar(500)")
            .IsRequired(false);
    }
}
```

MAPEIA UMA **STRING**. É ESSENCIAL
SEMPRE INFORMAR O TIPO
(TAMANHO) E SE O **CAMPO** É
OBRIGATÓRIO OU NÃO

MAPEANDO AS PROPRIEDADES

```
public class SignaturesMap : IEntityTypeConfiguration<Signature>
{
    public void Configure(EntityTypeBuilder<Signature> builder)
    {
        builder.ToTable("Signatures");
        builder.HasKey(s => s.Id);

        builder.Property(s => s.ContractId).IsRequired();
        builder.Property(s => s.Description).HasColumnType("varchar(100)").IsRequired();
        builder.Property(s => s.Status)
            .HasConversion(new EnumToStringConverter<ESignatureStatus>())
            .HasColumnType("varchar(30)")
            .IsRequired();
        builder.Property(s => s.HolderId).IsRequired();
        builder.Property(s => s.ExternalSignatureNumber).HasColumnType("varchar(100)").IsRequired(false);
        builder.Property(b => b.BankData)
            .HasConversion(
                c => c == null ? null : CryptoHelper.TripleDesEncrypt(c.ToJson()),
                s => string.IsNullOrEmpty(s) ? null! : CryptoHelper.TripleDesDecrypt(s).ToObject<BankData>())
            .HasColumnType("varchar(500)")
            .IsRequired(false);
    }
}
```

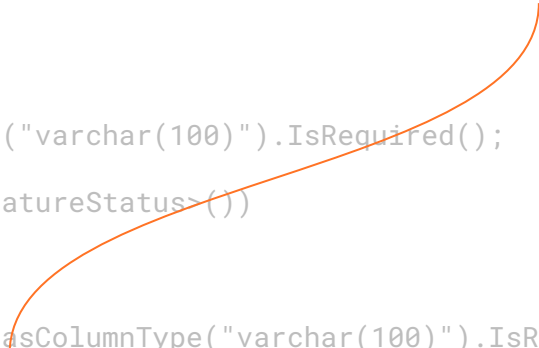
UTILIZA UM **CONVERSOR** PADRÃO DO **EF CORE** PARA **ENUM**, DE FORMA COM QUE ELE PERSISTA A STRING E NÃO O VALOR INTEIRO

MAPEANDO AS PROPRIEDADES

```
public class SignaturesMap : IEntityTypeConfiguration<Signature>
{
    public void Configure(EntityTypeBuilder<Signature> builder)
    {
        builder.ToTable("Signatures");
        builder.HasKey(s => s.Id);

        builder.Property(s => s.ContractId).IsRequired();
        builder.Property(s => s.Description).HasColumnType("varchar(100)").IsRequired();
        builder.Property(s => s.Status)
            .HasConversion(new EnumToStringConverter<ESignatureStatus>())
            .HasColumnType("varchar(30)")
            .IsRequired();
        builder.Property(s => s.HolderId).IsRequired();
        builder.Property(s => s.ExternalSignatureNumber).HasColumnType("varchar(100)").IsRequired(false);
        builder.Property(b => b.BankData)
            .HasConversion(
                c => c == null ? null : CryptoHelper.TripleDesEncrypt(c.ToJson()),
                s => string.IsNullOrEmpty(s) ? null! : CryptoHelper.TripleDesDecrypt(s).ToObject<BankData>())
            .HasColumnType("varchar(500)")
            .IsRequired(false);
    }
}
```

UTILIZA UM **CONVERSOR**
CUSTOMIZADO PARA REALIZAR A
CRİPTOGRAFIA NA ESCRITA E A
DESCRIPTOGRAFIA NA LEITURA



MAPEANDO AS PROPRIEDADES

r	BankData
	aDxn71VZEB/9FeR3WAxiuZ5DKugq1flcnw1ZX5ZGqZr29Cl4h4rBxEOKIDs4N+z6PazzFwCkdt6n4l00hijRVKP0nCsTJv3rXCJrUQydLs=

MAPEANDO AS PROPRIEDADES

A **SIGNATURE** POSSUI UMA
(**OWNSONE**) **VALIDITY**

```
builder.OwnsOne(c => c.Validity, validity =>
{
    validity
        .Property(v => v.StartDate)
        .HasColumnName("ValidityStartDate")
        .HasColumnType("datetime")
        .IsRequired();

    validity
        .Property(v => v.EndDate)
        .HasColumnName("ValidityEndDate")
        .HasColumnType("datetime")
        .IsRequired();
});
```

FAZ O **MAPEAMENTO** DAS
PROPRIEDADES DO **VALUE**
OBJECT

MAPEANDO AS PROPRIEDADES

```
builder
    !.HasMany(s => s.Beneficiaries)
    !.WithOne()
    !.HasForeignKey("SignatureId")
    !.OnDelete(DeleteBehavior.Cascade)
    !.IsRequired()
    !.Metadata
    !.PrincipalToDependent
    !.SetField("_beneficiaries");
```

MAPEANDO AS PROPRIEDADES

CONFIGURA UMA **RELAÇÃO** DE
UM-PARA-MUITOS

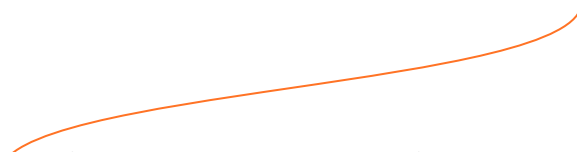
```
builder
    !.HasMany(s => s.Beneficiaries)
    !.WithOne()
    !.HasForeignKey("SignatureId")
    !.OnDelete(DeleteBehavior.Cascade)
    !.IsRequired()
    !.Metadata
    !.PrincipalToDependent
    !.SetField("_beneficiaries");
```

CONFIGURA UMA **RELAÇÃO 1XN**
ENTRE **SIGNATURE** E
BENEFICIARIES

MAPEANDO AS PROPRIEDADES

CONFIGURA O **LADO OPOSTO** DA RELAÇÃO,
INDICANDO QUE **CADA BENEFICIARY** ESTÁ
RELACIONADO A **UMA ÚNICA SIGNATURE**

```
builder
    !.HasMany(s => s.Beneficiaries)
    !.WithOne()
    !.HasForeignKey("SignatureId")
    !.OnDelete(DeleteBehavior.Cascade)
    !.IsRequired()
    !.Metadata
    !.PrincipalToDependent
    !.SetField("_beneficiaries");
```



MAPEANDO AS PROPRIEDADES

CONFIGURA A FK NA TABELA
FILHA

```
builder
  !.HasMany(s => s.Beneficiaries)
  !.WithOne()
  !.HasForeignKey("SignatureId")
  !.OnDelete(DeleteBehavior.Cascade)
  !.IsRequired()
  !.Metadata
  !.PrincipalToDependent
  !.SetField("_beneficiaries");
```



MAPEANDO AS PROPRIEDADES

QUANDO A **ENTIDADE PRINCIPAL** FOR **DELETADA**, TODAS AS **FILHAS VINCULADAS** **SERÃO TAMBÉM**

```
builder
    !.HasMany(s => s.Beneficiaries)
    !.WithOne()
    !.HasForeignKey("SignatureId")
    !.OnDelete(DeleteBehavior.Cascade)
    !.IsRequired()
    !.Metadata
    !.PrincipalToDependent
    !.SetField("_beneficiaries");
```



MAPEANDO AS PROPRIEDADES

METADADOS DA CONFIGURAÇÃO DA ENTIDADE

```
builder
    !.HasMany(s => s.Beneficiaries)
    !.WithOne()
    !.HasForeignKey("SignatureId")
    !.OnDelete(DeleteBehavior.Cascade)
    !.IsRequired()
    !.Metadata
    !.PrincipalToDependent
    !.SetField("_beneficiaries");
```

CONFIGURAÇÃO DE DETALHES ADICIONAIS DA NAVEGAÇÃO DA ENTIDADE PRINCIPAL PARA A FILHA

MAPEANDO AS PROPRIEDADES

```
builder
    !.HasMany(s => s.Beneficiaries)
    !.WithOne()
    !.HasForeignKey("SignatureId")
    !.OnDelete(DeleteBehavior.Cascade)
    !.IsRequired()
    !.Metadata
    !.PrincipalToDependent
    !.SetField("_beneficiaries");
```

CONFIGURA QUE O **BACKING FIELD** VAI ARMAZENAR A
LISTA DE **BENEFICIÁRIOS** DA **ASSINATURA**

UTILIZANDO STRUCT COMO VALUE OBJECT



UTILIZANDO STRUCT COMO VALUE OBJECT

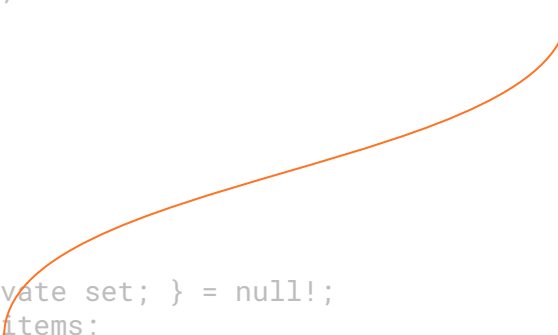
```
public partial class Order : AggregateRoot<int>
{
    private readonly ICollection<Item> _items = [];

    private Order() { }

    public Order(
        int id,
        IOrderState state,
        ICollection<Item> items,
        PaymentMethodInfo paymentMethod,
        DateTime date) : base(id)
    {
        State = state;
        _items = items ?? [];
        PaymentMethod = paymentMethod;
        Date = date;
    }

    public IOrderState State { get; private set; } = null!;
    public IEnumerable<Item> Items => _items;
    public PaymentMethodInfo PaymentMethod { get; private set; } = default;
    public DateTime Date { get; }
    public decimal Amount => _items.Sum(i => i.Amount);
    public string StateName => State.Name;
}
```

ESSA **PROPRIEDADE** É UM
VALUE OBJECT



UTILIZANDO STRUCT COMO VALUE OBJECT

```
public struct PaymentMethodInfo
{
    public PaymentMethodInfo(EPaymentMethod method, int? installments, bool wasRefunded)
    {
        Method = method;
        Installments = installments;
        WasRefunded = wasRefunded;
    }

    public EPaymentMethod Method { get; }
    public int? Installments { get; }
    public bool WasRefunded { get; private set; }

    public static PaymentMethodInfo Create(EPaymentMethod method, int? installments)
        => new PaymentMethodInfo(method, installments, wasRefunded: false);

    public void Refund() => WasRefunded = true;
}
```

UTILIZANDO STRUCT COMO VALUE OBJECT

```
public class OrderEMap : IEntityTypeConfiguration<Order>
{
    public void Configure(EntityTypeBuilder<Order> builder)
    {
        builder.ComplexProperty(o => o.PaymentMethod, paymentMethod =>
        {
            paymentMethod
                .Property(p => p.Method)
                .HasColumnName("PaymentMethod")
                .HasConversion(new EnumToStringConverter<EPaymentMethod>())
                .HasColumnType("varchar(20)")
                .IsRequired();

            paymentMethod
                .Property(p => p.Installments)
                .HasColumnName("PaymentMethodInstallments")
                .HasColumnType("int")
                .IsRequired(false);

            paymentMethod
                .Property(p => p.WasRefunded)
                .HasColumnName("PaymentMethodWasRefunded")
                .HasColumnType("bit")
                .IsRequired();
        });

        // Demais mapeamentos
    }
}
```

CAPÍTULO 5

ERROS COMUNS AO SE TRABALHAR COM ENTITY FRAMEWORK



ERROS COMUNS AO TRABALHAR COM EF CORE

6 references | 0 exceptions, - live

```
public async Task<IEnumerable<T>> GetAllAsync(int Id, CancellationToken cancellationToken = default)
{
    return await _context
        .Include(x => x.Address)
        .Include(x => x.PhoneNumber)
        .Include(x => x.FirstName)
        .Include(x => x.LastName)
        .Include(x => x.RegistrationNumber)
        .Include(x => x.RegistrationDate)
        .Include(x => x.RegistrationStatus)
        .Include(x => x.RegistrationType)
        .Include("Registration")
        .Include(a => a.Address)
        .Include(a => a.PhoneNumber)
        .Include("Registration")
        .Include(a => a.Address)
        .Include(a => a.PhoneNumber)
        .Include(a => a.RegistrationNumber)
        .Include(a => a.RegistrationDate)
        .Include(a => a.RegistrationStatus)
        .Include(a => a.RegistrationType)
        .Include(a => a.RegistrationStatus)
        .FirstOrDefaultAsync(x => x.RegistrationNumber == Id, cancellationToken);
}
```

ERROS COMUNS AO TRABALHAR COM EF CORE - USO ABUSIVO DE INCLUDES

CADA **INCLUDE** CONSTITUI UM **JOIN**

PROBLEMA DE **PERFORMANCE**, DADA A **COMPLEXIDADE** DA **QUERY** GERADA

POSSÍVEL **PROBLEMA** DE **DISPONIBILIDADE**, DADO A GRANDE **QUANTIDADE** DE **DADOS** ALOCADA EM **MEMÓRIA**

QUERIES MUITO **COMPLEXAS** PODEM OCASIONAR **PROBLEMAS** DE **TIME OUT**

ERROS COMUNS AO TRABALHAR COM EF CORE

```
public async Task<IEnumerable<Post>> GetByTitleAsync(string title, CancellationToken cancellationToken)
{
    return await _dbContextAccessor
        .Get()
        .Posts
        .Where(p => p.Title == title)
        .ToListAsync(cancellationToken)
        .ConfigureAwait(false);
}
```


ERROS COMUNS AO TRABALHAR COM EF CORE

SÃO **RETORNADOS TODOS** OS CAMPOS NO SELECT

```
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (525ms) [Parameters=[@__title_0='Test' (Size = 4000)], CommandType='Text', CommandTimeout='30']
      SELECT [p].[Id], [p].[AcceptedAnswerId], [p].[AnswerCount], [p].[Body], [p].[ClosedDate], [p].[CommentCount], [p].[CommunityOwnedDate], [p].[CreationDate], [p].[FavoriteCount], [p].[LastActivityDate], [p].[LastEditDate], [p].[LastEditorDisplayName], [p].[LastEditorUserId], [p].[OwnerUserId], [p].[ParentId], [p].[PostTypeId], [p].[Score], [p].[Tags], [p].[Title], [p].[ViewCount]
      FROM [Posts] AS [p]
      WHERE [p].[Title] = @__title_0
```

ERROS COMUNS AO TRABALHAR COM EF CORE

```
public async Task<IEnumerable<PostEfDto>> GetByTitleSimplifiedAsync(string title, CancellationToken
cancellationToken)
{
    return await _dbContextAccessor
        .Get()
        .Posts
        .Where(p => p.Title == title)
        .Select(p => new PostEfDto(p.Id, p.Title, p.Body))
        .ToListAsync(cancellationToken)
        .ConfigureAwait(false);
}
```

ERROS COMUNS AO TRABALHAR COM EF CORE

SÃO **RETORNADOS APENAS** OS
CAMPOS DESEJADOS

```
info: Microsoft.EntityFrameworkCore.Database.Command[20101]  
      Executed DbCommand (326ms) [Parameters=[@__title_0='Test' (Size = 4000)], CommandType='Text', CommandTimeout='30']  
      SELECT [p].[Id], [p].[Title], [p].[Body]  
      FROM [Posts] AS [p]  
      WHERE [p].[Title] = @__title_0
```

CAPÍTULO 6

CONSIDERAÇÕES FINAIS



CONSIDERAÇÕES IMPORTANTES AO TRABALHAR COM EF CORE

```
public async Task<IEnumerable<PostEfDto>> GetByTitleSimplifiedAsync(string title, CancellationToken
cancellation_token)
{
    return await _dbContextAccessor
        .Get()
        .Posts
        .TagWith("Query tag test!")
        .Where(p => p.Title == title)
        .Select(p => new PostEfDto(p.Id, p.Title, p.Body))
        .ToListAsync(cancellation_token)
        .ConfigureAwait(false);
}
```

```
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (366ms) [Parameters=[@__title_0='Test' (Size = 4000)], CommandType='Text', CommandTimeout='30']
      -- Query tag test!

      SELECT [p].[Id], [p].[Title], [p].[Body]
      FROM [Posts] AS [p]
      WHERE [p].[Title] = @__title_0
```

CONSIDERAÇÕES IMPORTANTES AO TRABALHAR COM EF CORE

```
public async Task<IEnumerable<PostEfDto>> GetByTitleSimplifiedAsync(string title, CancellationToken
cancellation_token)
{
    return await _dbContextAccessor
        .Get()
        .Posts
        .TagWithCallSite()
        .Where(p => p.Title == title)
        .Select(p => new PostEfDto(p.Id, p.Title, p.Body))
        .ToListAsync(cancellation_token)
        .ConfigureAwait(false);
}
```

```
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (406ms) [Parameters=[@__title_0='Test' (Size = 4000)], CommandType='Text', CommandTimeout='30']
      -- File: D:\Projects\Eximia.Course.AccessData\Eximia.Course.AccessData.API\Orders\Repositories\Posts\PostRepository.cs:32
```

```
SELECT [p].[Id], [p].[Title], [p].[Body]
FROM [Posts] AS [p]
WHERE [p].[Title] = @__title_0
```

CONSIDERAÇÕES IMPORTANTES AO TRABALHAR COM EF CORE

TRAZ PRA **MEMÓRIA** TODOS OS DADOS
PARA **DEPOIS FILTRAR**

```
public async Task<bool> HasAnyByTypeAsync(int typeId, CancellationToken cancellationToken)
{
    var posts = await _dbContextAccessor
        .Get()
        .Posts
        .Where(p => p.PostTypeId == typeId)
        .ToListAsync(cancellationToken)
        .ConfigureAwait(false);

    return posts.Count > 0;
}
```

CONSIDERAÇÕES IMPORTANTES AO TRABALHAR COM EF CORE

O **ANY()** FAZ A QUERY DE MODO MAIS **PERFORMÁTICO**, SEM RECUPERAR TODAS AS COLUNAS

```
public async Task<bool> HasAnyByTypeBetterAsync(int typeId, CancellationToken cancellationToken)
{
    return = await _dbContextAccessor
        .Get()
        .Posts
        .AnyAsync(p => p.PostTypeId == typeId)
        .ConfigureAwait(false);
}
```


CLUBE DE ESTUDOS
BY ELEMAR JR 

ELEMAR